АВОР

Agent Build & Operations Platform

Руководство администратора

Как развернуть АВОР у себя, подключить свои системы, завести данные и навыки, раздать доступы и держать платформу под контролем.

версия платформы **1.1** | навыков **58** | адаптеров данных **13** | дата **30.09.2026**

> Все команды в документе проверены на работающей установке. Пути и имена томов указаны точно: они важны — часть данных живёт только в томе, и запуск контейнера без него приводит к потере канонических записей.

СОДЕРЖАНИЕ

1 Состав поставки и топология
2 Требования и подготовка
3 Установка
4 Переменные окружения
5 Авторизация, роли и доступ
6 Модель: свой бокс или облако
7 Реестр систем и коннекторы
8 Data Plane: рецепты и сущности
9 Навыки: файлы, шаблоны, контракты
10 Агенты и семьи
11 Лимиты и бюджеты
12 Подбор исполнителя: агент или цепочка навыков
13 Очередь, группы и веер
14 Шина и коннектор-воркер
15 Доставка и шаблоны отчётов
16 Аудит и безопасность
17 Наблюдаемость
18 Биллинг и квоты
19 Резервное копирование и обновление
20 Диагностика
21 Карта процессов: редакции, области, подтверждение
22 Приложение А. Эндпоинты API
23 Приложение Б. Переменные окружения
24 Приложение В. Порты

РАЗДЕЛ 1

Состав поставки и топология

АВОР ставится одним контейнером на сервер заказчика. Всё остальное — опционально и включается по мере надобности.

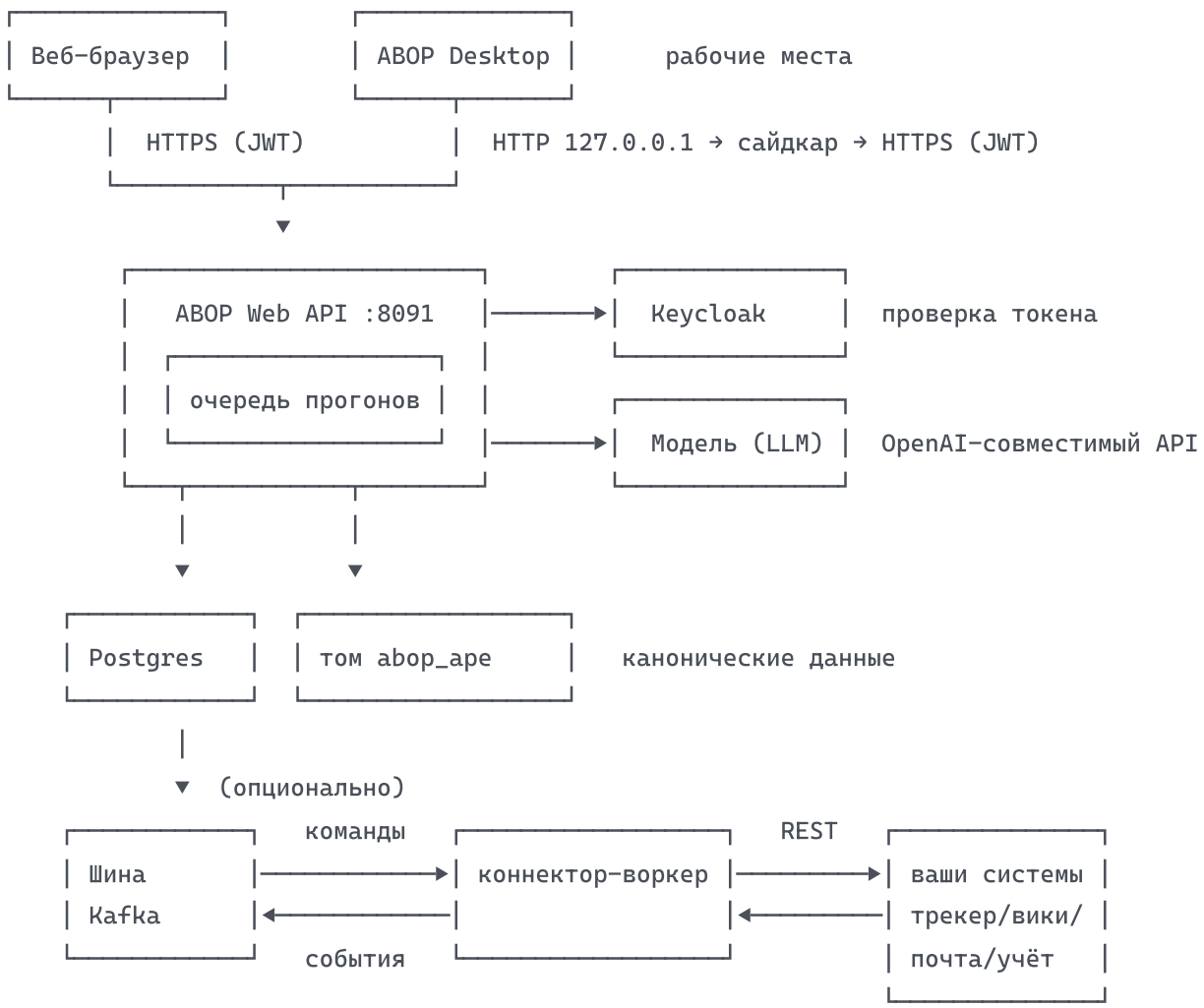
1.1 Обязательный минимум

КОМПОНЕНТ	ЧТО ДЕЛАЕТ	ГДЕ РАБОТАЕТ
АВОР Web API	Ядро: REST и SSE для веба, десктопа и агентов; внутри процесса — воркеры очереди прогонов	docker, порт 8091
Postgres	Агенты, контракты, прогоны, очередь заданий, шаблоны навыков и отчётов, рецепты, доступы, память, заявки на подтверждение	docker или внешний, порт 5432/5433
Том данных `abop_ape`	Канонические записи Data Plane (`/root/.ape/data/*.jsonl`)	docker volume
Модель	LLM с OpenAI-совместимым API: свой бокс с vLLM или облачный шлюз	свой сервер или внешний

1.2 Опциональные компоненты

КОМПОНЕНТ	КОГДА НУЖЕН
Keycloak	Всегда в продуктивной установке: без него аутентификация не настроена и API отвечает 401. Может быть ваш существующий
Шина (Kafka/Redpanda)	Когда нужны события из систем как триггеры и асинхронная доставка команд
Коннектор-воркер	Исполняет команды в ваших системах (создать задачу, отправить письмо, опубликовать страницу)
Prometheus + Grafana	Наблюдаемость: прогоны, очередь, стоимость, шина
Сервис HTML→PDF	Отчёты в PDF
Граф-RAG	Навыки, обязанные ссылаться на нормы и регламенты
S3-хранилище	Хранение отчётов и больших выгрузок

1.3 Как это связано



КЛЮЧЕВОЙ ПРИНЦИП

АВОР ничего не отправляет в ваши системы сам. Он формирует команду, показывает человеку, что именно уйдёт, и только после подтверждения публикует её. Исполняет команду отдельный воркер, у которого есть доступ к системам, — у ядра этих доступов нет.

РАЗДЕЛ 2

Требования и подготовка

2.1 Сервер приложения

ОС	Linux x86-64 с Docker 24+ (проверено на Ubuntu 22.04/24.04)
Процессор	4 ядра — минимум; 8 ядер — комфортно при 20+ пользователях
Память	4 ГБ минимум. Контейнеру ядра выделяется 1,5 ГБ: этого хватает на 4 воркера прогонов
Диск	20 ГБ на систему и образы, плюс объём под данные Postgres и канонический слой
Сеть	Исходящий доступ к модели; входящий — только к порту веб-интерфейса, желательно за обратным прокси с TLS

2.2 Сервер модели

Модель — отдельная история. Есть два пути.

ПУТЬ	ЧТО НУЖНО	КОГДА ВЫБИРАТЬ
Свой бокс	GPU от 48 ГБ памяти (L40S, A6000, 6000 Ada), vLLM, модель уровня Qwen3-30B-A3B в FP8	Данные не должны покидать контур; нужен предсказуемый расход
Облачный шлюз	Только ключ доступа к OpenAI-совместимому API	Быстрый старт, пилот, небольшие объёмы

РЕКОМЕНДАЦИЯ ПО МОДЕЛИ

Для продуктивной работы берите модель с поддержкой строгого структурного вывода (`response_format: json_schema`). АВОР опирается на схемы результата: если модель умеет соблюдать схему на уровне декодирования, качество и стоимость заметно лучше, чем при разборе «свободного» ответа.

2.3 Что подготовить заранее

- **Учётные записи** в системах, из которых агенты будут читать данные, — с правами только на чтение;
- **Отдельные учётные записи** для систем, в которые агенты будут писать (трекер, вики, почта), — чтобы действия агентов отличались от действий людей;
- **Список отделов** и того, какие системы каждому доступны: это ляжет в основу правил доступа;
- **Realm в Keycloak** (или доступ к существующему) с ролями `admin` , `support` , `manager` , `analyst` .

РАЗДЕЛ 3

Установка

Базовая установка занимает около получаса: разложить файлы, заполнить окружение, собрать образ, запустить, проверить.

1 Разложите поставку

Каталог `/opt/abop`. В нём должны оказаться `server/`, `cli/`, `skills/`, `webapp/`, `reports/`, `desktop/ui/`, `ops/` и `Dockerfile.webapi`.

```
mkdir -p /opt/abop && cd /opt/abop
tar -xf abop-<версия>.tar
```

2 Поднимите Postgres

Можно взять существующий. Для отдельного:

```
docker run -d --name abop-pg --restart unless-stopped \
  -e POSTGRES_PASSWORD='…' -e POSTGRES_DB=abop -e POSTGRES_USER=abop \
  -p 127.0.0.1:5433:5432 -v ape_pg:/var/lib/postgresql/data postgres:16
```

Порт наружу не открывайте: ядро ходит в базу по петле.

3 Заполните `.env`

Минимальный рабочий набор — в разделе 4. Файл лежит рядом: `/opt/abop/.env`, права `600`.

4 Соберите и запустите

```
cd /opt/abop && bash ops/deploy-webapi.sh
```

Скрипт собирает образ, пересоздаёт контейнер с нужным томом и ждёт готовности.

Эквивалент вручную:

```
docker build -q -f Dockerfile.webapi -t abop-webapi .
docker rm -f abop-webapi 2>/dev/null || true
docker run -d --name abop-webapi --network host --restart unless-stopped --
memory=1536m \
  --env-file /opt/abop/.env -v abop_ape:/root/.ape abop-webapi
```

5 Проверьте

```
curl -s http://127.0.0.1:8091/api/health
# {"ok":true,"families":10,"skills":58,"adapters":[...]}
```

Если `skills` равно нулю — каталог навыков не попал в образ.

6 Закройте периметр

Порт 8091 наружу отдавайте только через обратный прокси с TLS. Прямой доступ к Postgres, Keycloak и шине снаружи должен быть закрыт.

НИКОГДА НЕ ЗАПУСКАЙТЕ БЕЗ ТОМА

Флаг `-v abop_ape:/root/.ape` обязателен. В этом томе лежат канонические записи Data Plane. Контейнер, запущенный без него, стартует и выглядит здоровым, но все данные, выгруженные рецептами, пропадут при следующем пересоздании.

3.1 Первый вход

Откройте `https://<ваш-адрес>/`. Вход идёт через Keycloak: ABOP проксирует логин, поэтому сам Keycloak можно держать закрытым от внешней сети. После входа в правом нижнем углу появится ваше имя, уровень доступа и отдел.

> **РЕЖИМ РАЗРАБОТКИ**
>
> Переменная `ABOP_DEV_AUTH=1` отключает проверку токенов и делает любого анонима администратором. Она предназначена для локальной разработки и автотестов. В продуктивной установке её быть не должно — проверьте перед сдачей: `docker exec abop-webapi printenv | grep DEV_AUTH` не должен ничего вернуть.

3.2 Руководства внутри продукта

Оба руководства едут в образе и отдаются самой платформой: каталог `docs/guide` монтируется на `/guide`, список — `GET /api/guides`. Пользователь открывает их кнопкой «Помощь» в вебе и блоком «Руководства» в кабинете десктопа. Если каталог не вошёл в сборку, список пуст, а интерфейс честно говорит об этом вместо битых ссылок.

3.3 Установка десктоп-клиента

Клиент ставится на машины пользователей отдельно. Установщик собирается командой `npm run dist` в каталоге `desktop/` и публикуется вашим способом раздачи ПО. Клиенту нужно знать адрес сервера — он задаётся переменной окружения `ABOP_BASE_URL` при сборке или в конфигурации приложения.

Интерфейс десктопа подтягивается с сервера при каждом запуске (`/desktop-ui-bundle`), поэтому обновления интерфейса доезжают до пользователей без переустановки приложения — переустановка нужна только при смене версии оболочки или локального движка.

РАЗДЕЛ 4

Переменные окружения

Файл `/opt/abop/.env` — единственный источник настроек уровня среды. Полный список с умолчаниями — в приложении Б; здесь минимум, который нужно заполнить осознанно.

4.1 Минимальный рабочий набор

```
# ── база ──
POSTGRES_DSN=postgresql://abop:пароль@127.0.0.1:5433/abop

# ── авторизация ──
KEYCLOAK_ISSUER=https://auth.example.ru/realms/abop
KEYCLOAK_JWKS_URI=https://auth.example.ru/realms/abop/protocol/openid-connect/certs
KEYCLOAK_AUDIENCE=abop-web

# ── модель: свой бокс ──
LOCAL_LLM_BASE_URL=http://10.0.0.20:8000/v1
LOCAL_LLM_MODEL=qwen3-30b-a3b
LOCAL_LLM_API_KEY=EMPTY

# ── исполнение прогонов ──
ABOP_RUN_WORKERS=4
ABOP_RUN_TIMEOUT=600
ABOP_USER_CONCURRENT=1
ABOP_RUN_LLM_TRUNCATE=0
```

4.2 Что стоит поменять под нагрузку

ПЕРЕМЕННАЯ	ПО УМОЛЧ.	КОГДА МЕНЯТЬ
ABOP_RUN_WORKERS	2	Сколько прогонов исполняется одновременно. Каждый воркер — это ещё один параллельный поток запросов к модели. Поднимайте вместе с ёмкостью модели
ABOP_USER_CONCURRENT	1	Сколько прогонов одного пользователя идут одновременно. Единица защищает от того, что один человек займёт всю очередь
ABOP_RUN_TIMEOUT	600	Потолок на задание в секундах. Длинные аудиты по большим выгрузкам требуют больше
ABOP_RUN_LLM_CONCURRENCY	3	Сколько навыков внутри одного прогона идут в модель одновременно
ABOP_LLM_MAX_INFLIGHT	12	Общий потолок одновременных запросов к модели. Ставьте по ёмкости бокса
ABOP_RUN_LLM_TRUNCATE	1	**На проде поставьте 0.** Единица режет промпты и выборки для быстрой отладки
ABOP_RUN_CACHE	1	Кэш одинаковых прогонов. Ноль — всегда считать заново
ABOP_FANOUT_MAX	20	Максимум заданий в одном веере — защита от случайного запуска по двумстам предметам
ABOP_SCHEDULER	1	Планировщик регулярных запусков. Ноль — выключить

ЛИМИТЫ ПРОГОНА БОЛЬШЕ НЕ ЖИВУТ В ОКРУЖЕНИИ

Длина ответа навыка, шаги инструментов, размер выборки и параллельность настраиваются из интерфейса — **Настройки** → **Прогон** (раздел 11). Переменные окружения остались умолчаниями: настройка из интерфейса их перекрывает и действует со следующего прогона, без перезапуска.

РАЗДЕЛ 5

Авторизация, роли и доступ

АВОР не ведёт своих учётных записей. Пользователи и роли живут в вашем Keycloak, а платформа читает их из токена.

5.1 Настройка Keycloak

1. **Создайте realm**

   Например `abop`. Если у вас уже есть корпоративный realm — используйте его.

2. **Заведите клиента для веба**

   `abop-web`, тип «public», поток «Direct access grants» включён — через него АВОР проксирует вход, не выставляя Keycloak наружу.

3. **Заведите роли уровня realm**

   `admin`, `support`, `manager`, `analyst`.

4. **Добавьте атрибут отдела**

   Атрибут пользователя `department` со значением-идентификатором семьи (`management`, `finance`, `audit`…). Пропишите его в маппер токена, чтобы он попадал в JWT. Значение `*` означает «все области».

5. **Пропишите адреса в `.env`**

   `KEYCLOAK_ISSUER` и `KEYCLOAK_JWKS_URI`. Если Keycloak внутри той же сети, а issuer публичный, задайте `KEYCLOAK_JWKS_INTERNAL` — ключи будут забираться по внутреннему адресу, а issuer проверяться по публичному.

5.2 Четыре уровня доступа

УРОВЕНЬ	МОЖЕТ	НЕ МОЖЕТ
analyst аналитик	Смотреть агентов, навыки, данные и прогоны своей области. Запускать прогоны	Менять агентов, навыки, данные, настройки. Подтверждать внешние действия
manager менеджер	Всё, что аналитик, плюс собирать и публиковать агентов в своём отделе, подтверждать внешние действия, править лимиты прогона	Менять шаблоны навыков, управлять доступом, трогать чужие отделы
support поддержка	Сквозное чтение по всем отделам для разбора обращений	Менять что-либо
admin администратор	Всё: навыки, шаблоны, системы, доступы, удаление агентов	—

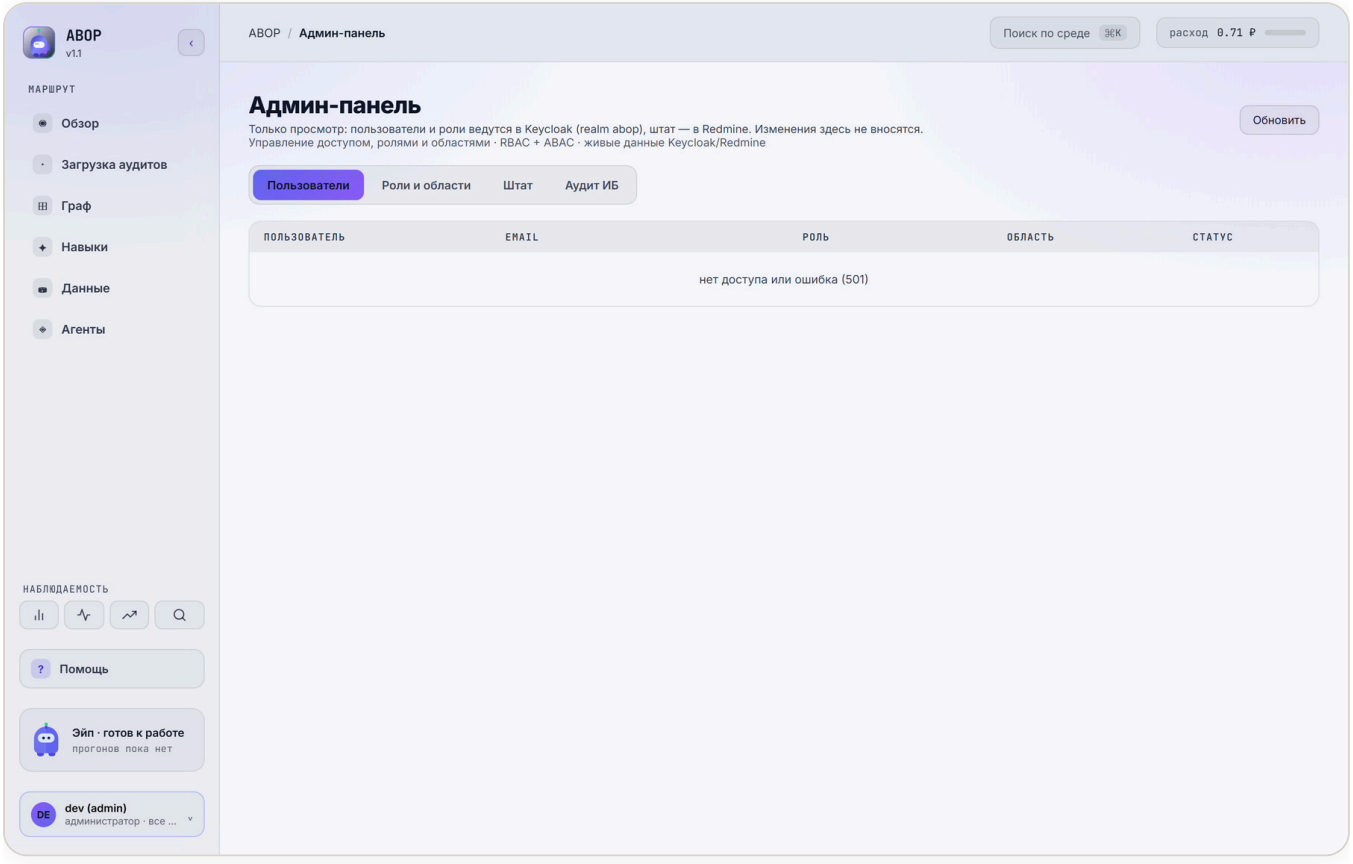


Рис. 5.1. Админ-панель, вкладка «Пользователи»: кто заведён, с каким уровнем и в какой области. Список читается из Keycloak.

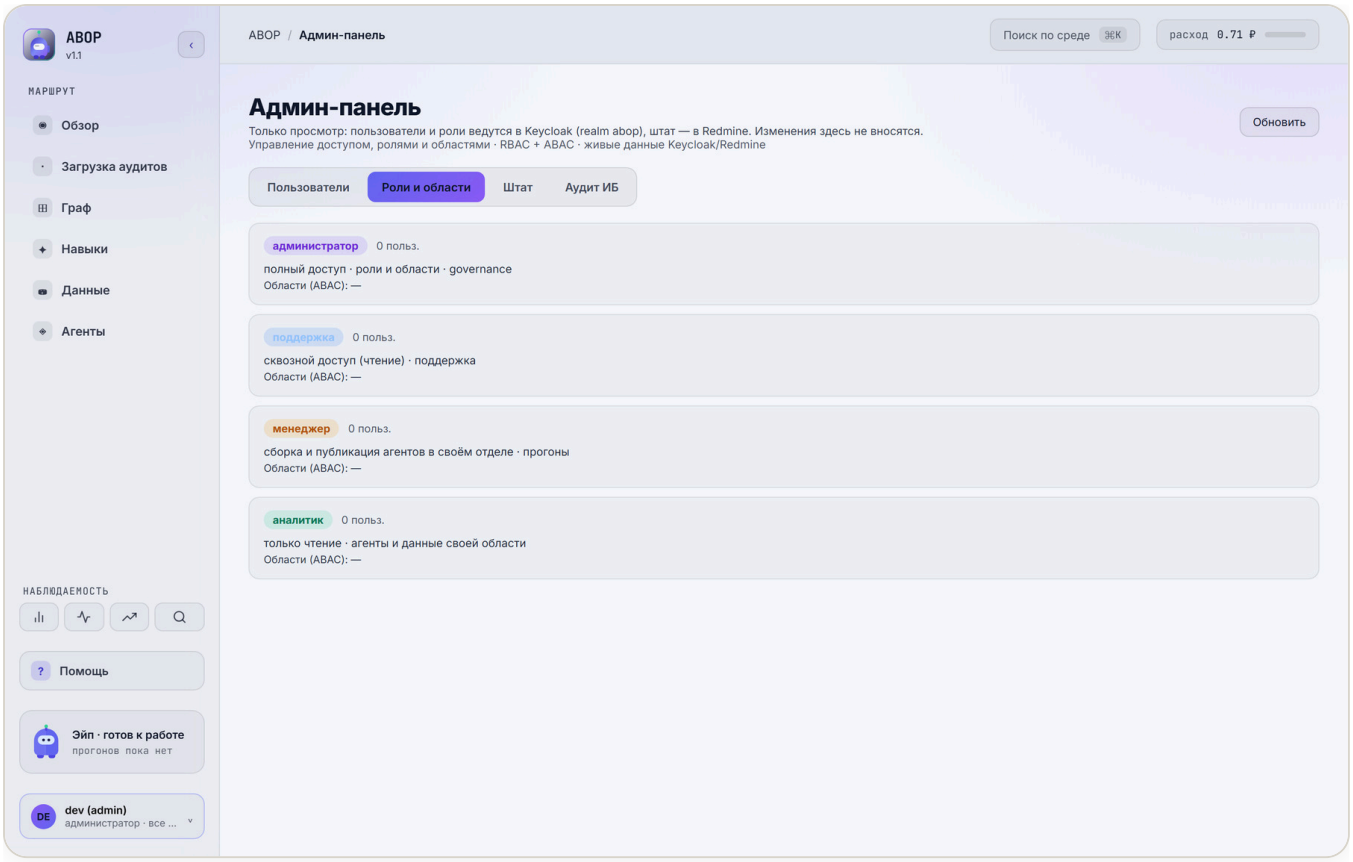


Рис. 5.2. «Роли и области»: сколько людей на каждом уровне, какие права он даёт и какие отделы за ним закреплены.

5.3 Доступ к данным: ABAC по отделу

Кроме уровня доступа, действует правило области. Агент принадлежит семье (отделу), а каждая система в реестре объявляет, каким семьям она доступна. Если сущность наполняется рецептом из системы, недоступной семье агента, — агент эту сущность не читает. В прогоне появляется явная пометка «доступ к данным запрещён», а отказ пишется в аудит.

> КАК ЭТО ПРОВЕРИТЬ
>
> Откройте результат прогона: блок «доступ» показывает, какие системы были разрешены агенту и по какой причине. Ничего не скрывается: видно и разрешённые, и закрытые.

5.4 Аудит

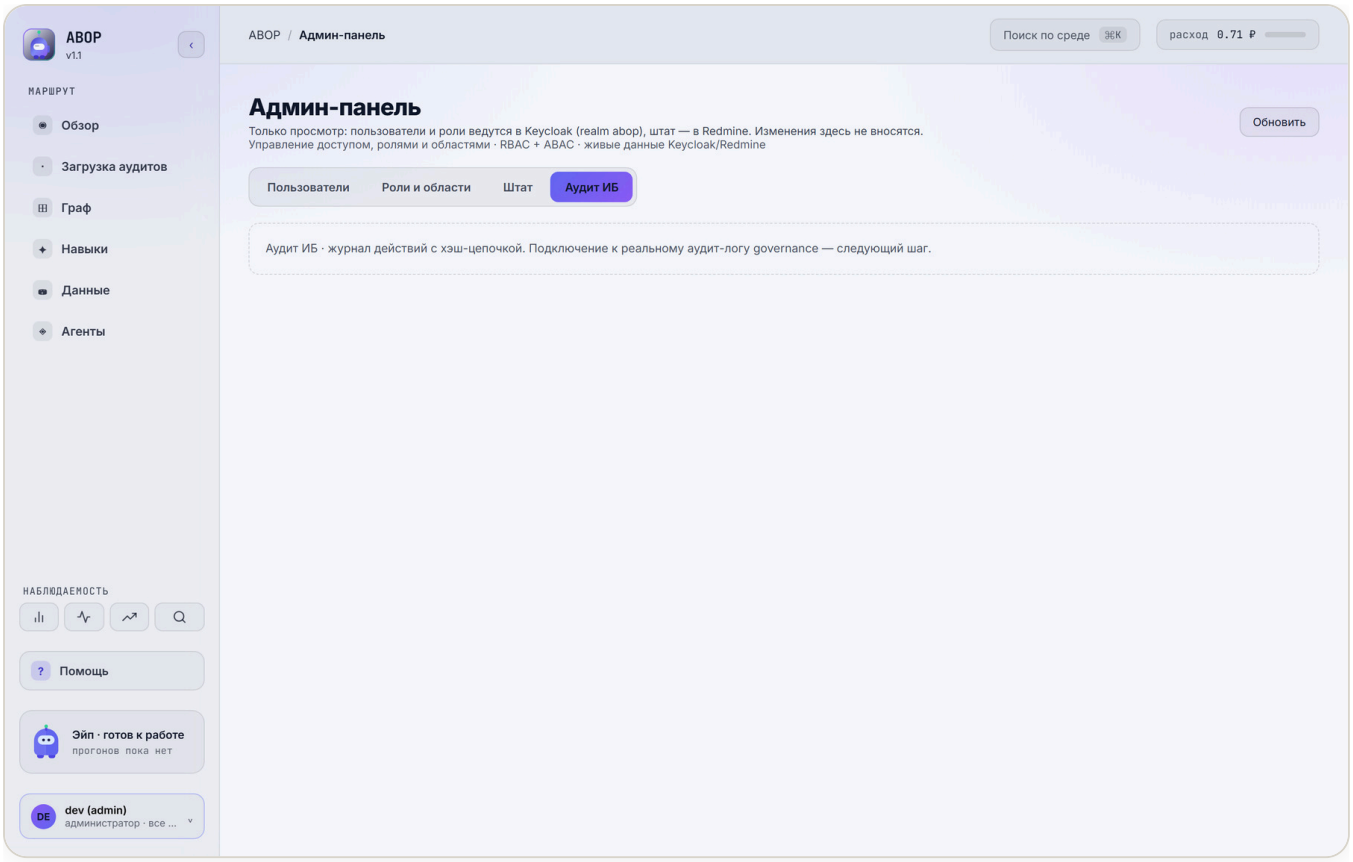


Рис. 5.3. Аудит-след: кто, что и когда сделал, с пометкой критичности.

В аудит попадают: запуски прогонов, сохранение и удаление агентов, правка навыков и шаблонов, изменение настроек и лимитов, подтверждения и отклонения внешних действий, отказы по правам доступа. Запись содержит исполнителя, объект, время и краткое описание.

РАЗДЕЛ 6

Модель: свой бокс или облако

АВОР работает с любым OpenAI-совместимым API. Адрес модели можно сменить на лету, не перезапуская сервис.

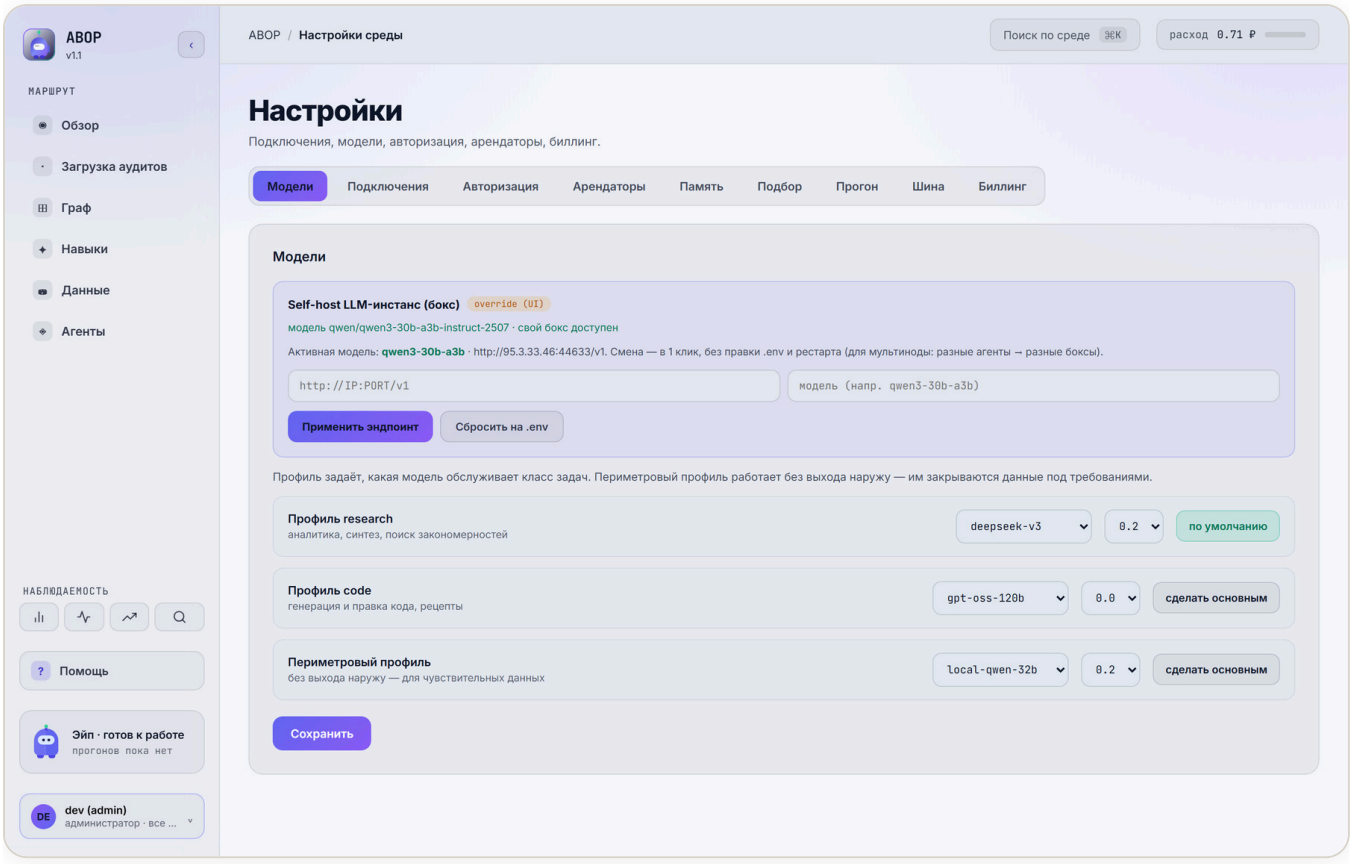


Рис. 6.1. «Настройки → Модели»: текущий адрес, модель и состояние проверки связи. Здесь же — переключение на запасной канал.

6.1 Свой бокс с vLLM

```
vllm serve Qwen/Qwen3-30B-A3B-Instruct-2507-FP8 \
  --port 8000 --max-model-len 32768 --gpu-memory-utilization 0.92 \
  --guided-decoding-backend xgrammar
```

Проверьте, что модель отвечает:

```
curl -s http://10.0.0.20:8000/v1/models
```

Затем пропишите адрес в `.env` (`LOCAL_LLM_BASE_URL`, `LOCAL_LLM_MODEL`) или задайте из интерфейса — «Настройки → Модели → Применить эндпоинт». Настройка из интерфейса переживает перезапуск контейнера: она хранится в базе.

6.2 Облачный шлюз

Задайте базовый адрес и ключ вашего провайдера. Каскад моделей описывается списком: если первая не отвечает, запрос уходит следующей. Откаты видны в метрике `abop_llm_fallback_total` — если она растёт, основная модель нестабильна.

6.3 Три профиля запроса

`standard`	Агентский движок: все прогоны навыков
`code`	Работа с кодом и таблицами
`research`	Длинные разборы и исследования

Каждому профилю можно назначить свой каскад — например, тяжёлую модель только для исследований.

6.4 Проверка после смены модели

1. `GET /api/llm/status` — адрес, активная модель, результат проверки связи;
2. Запустите короткий прогон с бюджетом «короткий» — убедитесь, что ответ приходит и соблюдает схему;
3. Посмотрите в результате прогона поле стоимости: если тариф для новой модели не задан, расход будет считаться нулевым.

> **СТРУКТУРНЫЙ ВЫВОД**
>
> Если модель не поддерживает `response_format: json_schema`, включите `ABOP_RUN_STRUCTURED=0`. Схема результата тогда проверяется после ответа: навык, не уложившийся в схему, будет переспрошен один раз, а если и это не помогло — в результате появится честная пометка о несоблюдении схемы вместо выдуманных полей.

РАЗДЕЛ 7

Реестр систем и коннекторы

Реестр систем — единый справочник того, куда АВОР вообще может ходить. Ни рецепт, ни доставка не работают с «просто адресом»: они ссылаются на систему из реестра.

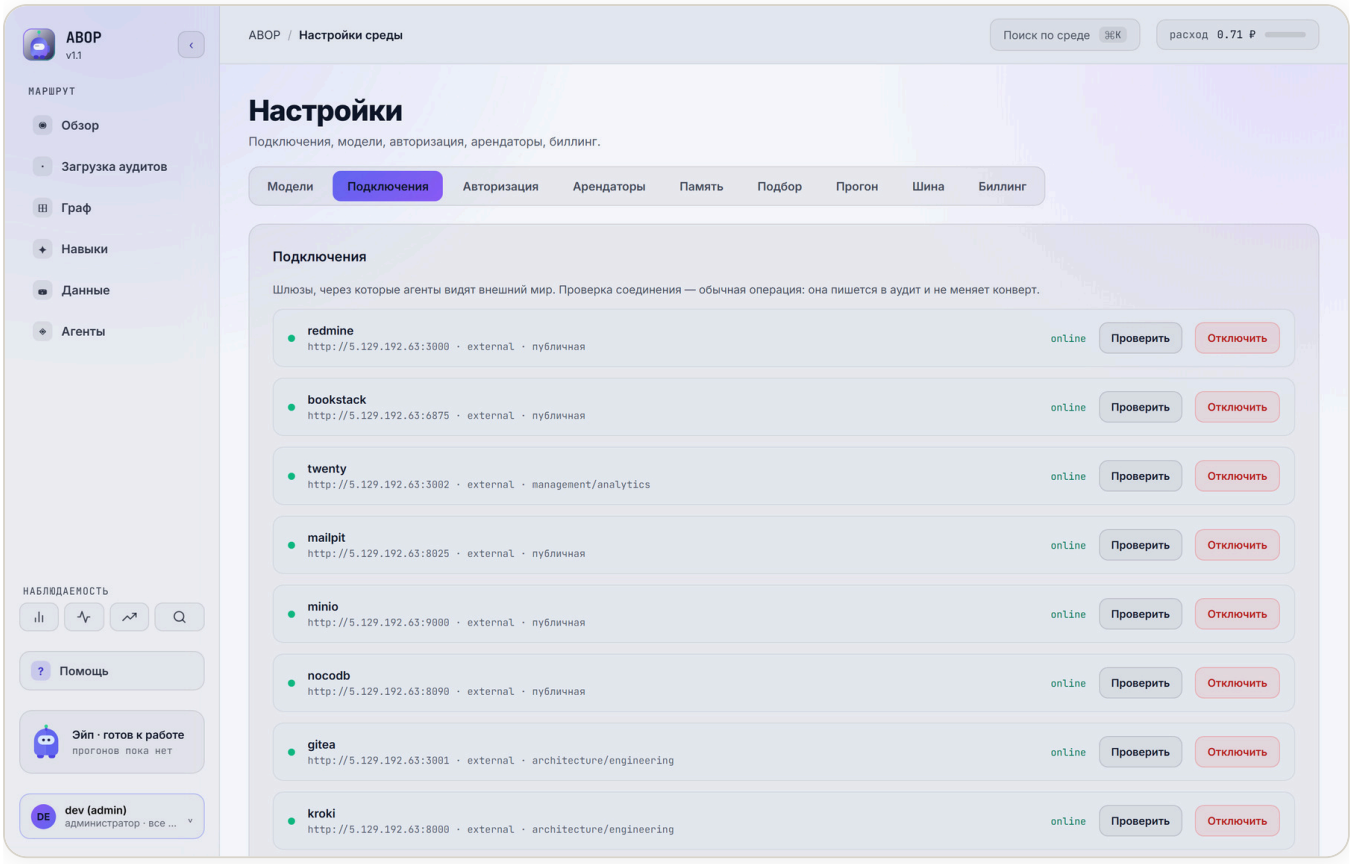


Рис. 7.1. «Настройки → Подключения»: системы реестра с адресами и состоянием.

7.1 Что описывает запись системы

Идентификатор	короткий slug: `redmine`, `bookstack`, `erp`
Тип	`rest`, `postgres`, `s3`, `smtp`, `kafka`
Адрес	базовый URL или строка подключения
Выход наружу	`internal` — внутри контура, `external` — за его пределами. Влияет на правила и на то, требуется ли подтверждение
Области (scope)	каким семьям система доступна. Пусто — доступна всем
Топики шины	если включена шина, системе выделяется пара топиков: события и команды

7.2 Заведение системы

1. **Добавьте запись в реестр**

   «Настройки → Подключения → добавить». Укажите slug, тип, адрес и области доступа.

2. **Создайте коннектор**

   «Данные → Коннекторы». Коннектор — это способ авторизации: токен, логин и пароль, заголовок. Секреты хранятся на сервере и в интерфейсе больше не показываются.

3. **Проверьте связь**

   Кнопка проверки делает настоящий запрос и показывает, сколько строк вернулось и какие колонки найдены. Если источник ответил, но строк ноль — скорее всего, данные лежат под другим ключом: укажите корневой путь.

4. **Закройте лишнее**

   Учётная запись коннектора должна иметь ровно те права, которые нужны: чтение — для источников, запись — только для систем доставки.

7.3 Защита от выхода не туда

Запросы наружу проходят через проверку адреса: запрещены обращения к внутренним и служебным подсетям, если система не объявлена внутренней. Это защита от того, что подставленный адрес в рецепте заставит платформу сходить по внутреннему адресу вашей сети.

РАЗДЕЛЬНЫЕ УЧЁТНЫЕ ЗАПИСИ

Не используйте одну учётную запись для чтения и записи. Если агент читает трекер и он же в нём создаёт задачи — ошибка в правиле доставки превращается в массовое создание задач под правами чтения-записи. Разделите: одна запись читает, другая пишет, и только в один проект.

РАЗДЕЛ 8

Data Plane: рецепты и сущности

Агент никогда не читает вашу систему напрямую. Между ними стоит рецепт: он забирает данные, приводит их к каноническому виду, проверяет и кладёт в хранилище с указанием происхождения. Навык объявляет сущность — и получает чистые записи, не зная, откуда они.

```
источник ⟶ маппинг и типизация ⟶ правила ⟶ canonical JSON ⟶ запрос навыка
 (CSV, REST,        (колонка → поле,      (проверки,      (data/<сущность>.jsonl   (data
  Postgres, S3)      приведение типа)      маскировка)      + провенанс)
```

8.1 Адаптеры источников

Поставка включает 13 адаптеров:

Файлы	`csv`, `json`, `xlsx`, `text`, `pdf`
Базы	`postgres`, `sqlite`
Сеть	`http` (REST), `mailpit` (почта), `vector` (векторное хранилище)
Специальные	`audit1c` (выгрузка учётной системы), `slava` (граф-RAG по нормам), `computed` (вычисляемая сущность)

8.2 Как устроен рецепт

```
{
  "recipe": "sales",
  "version": 1,
  "source": { "kind": "csv", "path": "/data/sales.csv" },
  "entity": "transaction",
  "system_id": "erp",
  "map": {
    "id":       { "col": "order_id" },
    "customer": { "col": "email", "cast": "lower" },
    "amount":   { "col": "total", "cast": "money" },
    "date":     { "col": "created_at" }
  },
  "rules": [
    { "assert": "amount > 0" },
    { "pii_mask": ["customer"] }
  ],
  "emit": { "schema": "transaction", "schema_version": "1.0", "ttl_sec": 86400 }
}
```

СЕКЦИЯ	НАЗНАЧЕНИЕ
`source`	откуда брать: адаптер и адрес. Для REST — ещё `root` (путь к массиву в ответе) и заголовки
`entity`	в какую каноническую сущность превращается строка
`system_id`	система реестра — по ней работают правила доступа отдела
`map`	колонка источника → поле сущности, с приведением типа (`money`, `lower`, `date`, `int`)
`rules`	проверки (`assert`) и маскировка персональных данных (`pii_mask`). Строка, не прошедшая проверку, отбрасывается с подсчётом
`emit`	имя схемы, версия и время жизни записи

8.3 Прогон рецепта

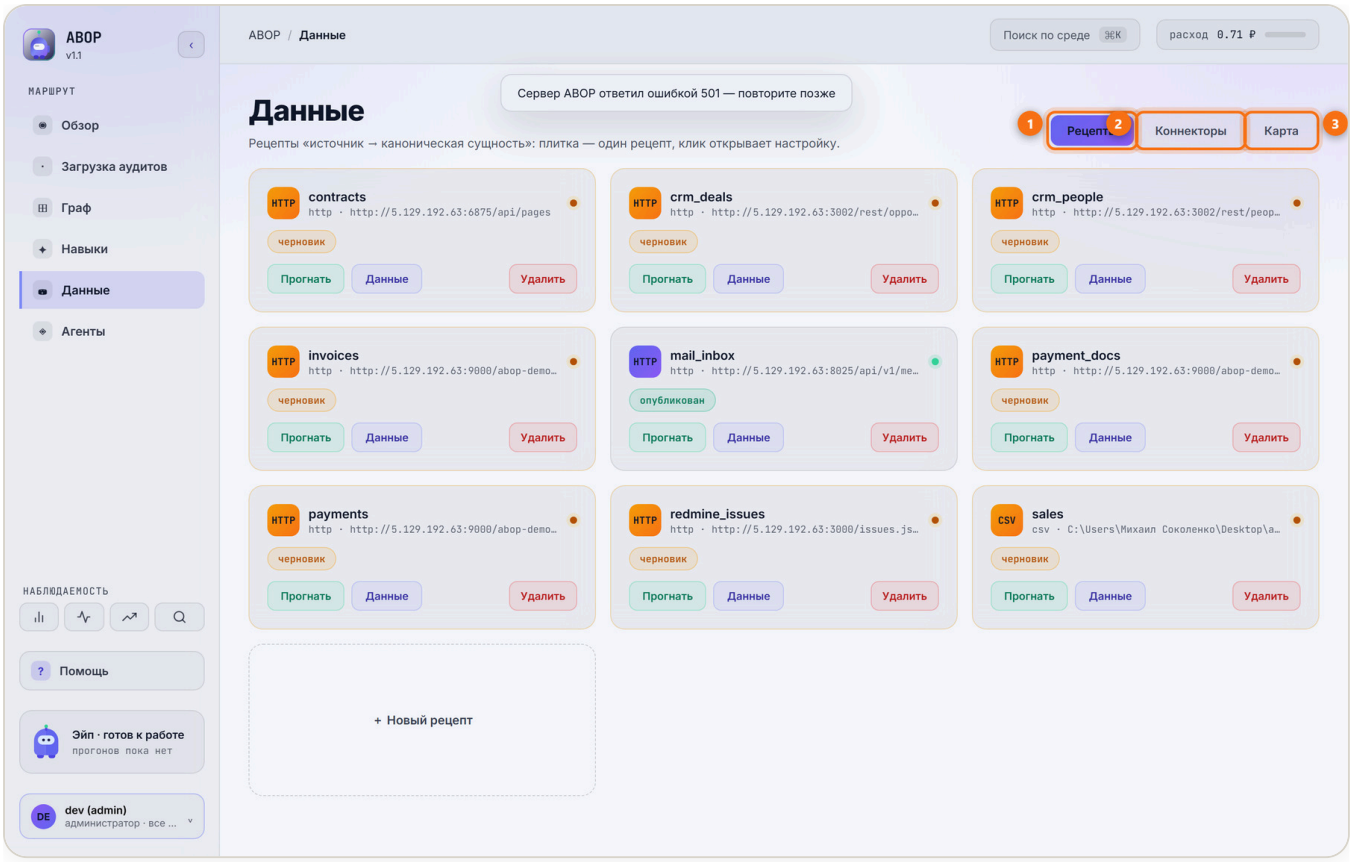


Рис. 8.1. Рецепты: адаптер, адрес источника, статус. Три действия у каждого — прогнать, посмотреть данные, удалить.

1. **Рецепты.** Правила «источник → сущность».
2. **Коннекторы.** Подключения к системам.
3. **Карта.** Что откуда наполняется и кто это читает.

Результат прогона рецепта — честная сводка: *записано / отброшено правилами / не прошло проверку схемы*. Если отброшено много — смотрите правила: скорее всего, источник изменил формат.

ДВЕ ЛОВУШКИ, НА КОТОРЫХ ТЕРЯЮТ ДАННЫЕ

«Начисто» действует на сущность, а не на рецепт. Если два рецепта пишут в одну сущность (например, платежи и счета — оба в `transaction`), начисто можно читать только первый, второй обязан идти дозаписью. Иначе второй прогон перепишет файл сущности своими строками, и вместо 180 записей останется 60. Заметно это только по счётчику записей в разделе «Данные» — ошибки не будет.

Рецепты перечитывайте через интерфейс или API, не скриптом мимо сервера. Рабочая версия рецепта лежит в Postgres; рядом может оставаться устаревшая файловая копия. Скрипт, запущенный в контейнере напрямую, берёт файл — и прогон отдаёт, например, «записано 0, не прошло проверку схемы 21», потому что в старой версии поле отправителя читалось из другой колонки источника. Правильно: `POST /api/data/recipes/{name}/run` или кнопка «Прогнать» в разделе «Данные».

8.4 Версии и время жизни записей

Хранилище append-only: повторный прогон рецепта не затирает старое, а добавляет новую версию записи. При чтении отдаётся последняя созданная версия. Важное свойство: **последняя версия не протухает по времени** — данные не исчезают сами. Время жизни (`ttl_sec`) вытесняет только перекрытые старые версии.

ПРАКТИКА

Ставьте `ttl_sec: 0` для справочников (они меняются редко) и сутки-двое для операционных данных. Регулярный прогон рецептов удобно повесить на планировщик через агента-обновлятора либо внешним cron через API.

8.5 Карта данных

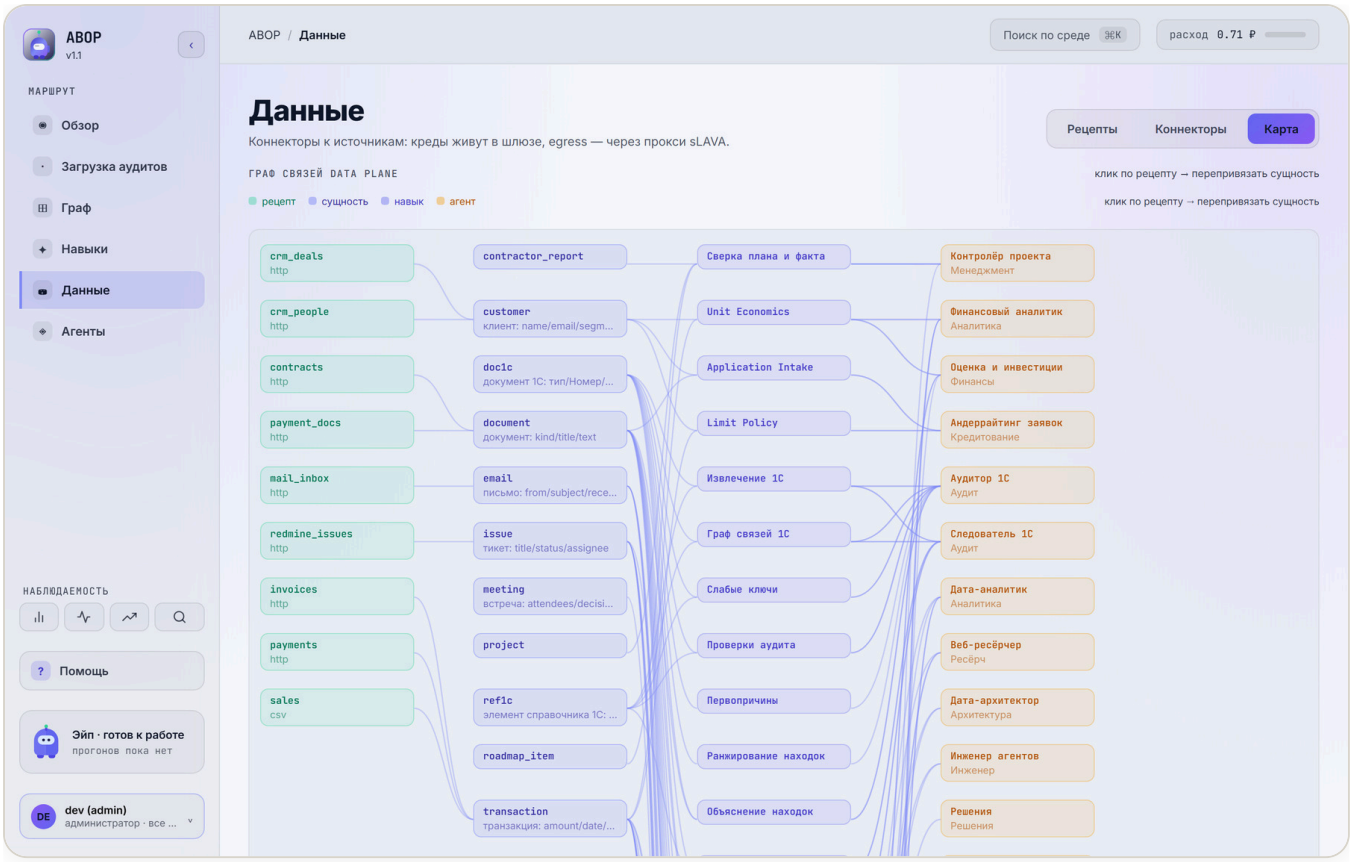


Рис. 8.2. Карта: сущности, их источники и навыки-потребители. Разрывы видны сразу — сущность объявлена навыком, но рецепта под неё нет.

Карта — первое место, куда стоит смотреть, когда прогоны возвращают ноль находок. Эндпоинт `GET /api/data/entities` отдаёт тот же список машинно: имя сущности, сколько записей лежит, сколько навыков её объявили.

РАЗДЕЛ 9

Навыки: файлы, шаблоны, контракты

Навык — это методика плюс контракт. Он живёт в двух файлах и в базе: файлы едут с поставкой, база — источник правды для исполнения.

9.1 Файлы навыка

```
skills/<идентификатор>/
  SKILL.md        # методика: что делать, когда применять, чего не делать
  template.json   # контракт: схема результата, вход, выход, лимиты, доставка
```

SKILL.md

```
---
name: acceptance-check
description: Смотрит на пункты проекта глазами приёмки заказчика…
tools: data_query, data_get, data_schema
---

# Приёмка заказчика по пунктам проекта

Ты отвечаешь на вопрос принимающей стороны: что из заявленного подрядчиком
заказчик действительно принял…

## Когда применять
## Метод
## Чего не делать
## Результат
```

Заголовок (frontmatter) объявляет идентификатор, назначение и инструменты, которыми навык может пользоваться. Тело — методика: она уходит в системное сообщение модели и определяет, как навык думает. Раздел «Чего не делать» не декоративный: он снимает типовые ошибки модели.

template.json

ПОЛЕ	НАЗНАЧЕНИЕ
`name`	человеческое название навыка
`instruction`	инструкция по заполнению результата: как считать, чего не смешивать
`json_schema`	строгая схема результата. Ответ проверяется по ней; не уложившийся навык переспрашивают
`slots`	предмет работы: какую сущность нужно уточнить у человека, по каким полям искать, как показывать
`inputs`	чем кормить: из данных, из навыка, из слота, из контекста, с доски прогона
`produces`	что отдаёт: какой раздел результата и по какому ключу он соединяется с другими
`delivery`	куда уходит результат: система, тип команды, шаблон полей
`max_tokens`	лимит ответа этого навыка
`tool_steps`	сколько раз навык может сходить в инструменты до ответа; ноль — не ходит

9.2 Контракт: вход и выход

```
"inputs": {
  "required": [
    { "from": "slot",  "name": "проект" },
    { "from": "data",  "entity": "roadmap_item",
      "fields": ["проект", "пункт", "план_финиш", "часы_план"] },
    { "from": "skill", "skill": "roadmap-fact", "path": "исполнение" },
    { "from": "board", "key": "исполнение" }
  ],
  "optional": [ … ]
},
"produces": [
  { "path": "исполнение", "key": "пункт", "join": "пункт_плана" }
]
```

ИСТОЧНИК ВХОДА	ЧТО ОЗНАЧАЕТ
`data`	записи канонической сущности, с перечнем нужных полей
`skill`	результат другого навыка — так собираются цепочки. Пока вход не готов, навык не запускается
`slot`	предмет работы, который уточняется у человека
`context`	формулировка задачи и приложенные файлы
`board`	вывод другой ветви с доски прогона по объявленному ключу

ЗАЧЕМ КОНТРАКТ

По контракту среда знает *до запуска*, исполнима ли цепочка: покрыты ли обязательные входы, есть ли поставщик нужного выхода. Без контракта это выяснялось бы на середине прогона — уже потратив деньги.

9.3 Добавление своего навыка

1 **Создайте каталог**

`skills/my-skill/` с двумя файлами. Идентификатор — латиницей, через дефис.

2 Опишите методику

В `SKILL.md`: когда применять, метод по шагам, чего не делать, что на выходе. Пишите так, как объясняли бы новому сотруднику.

3 Опишите контракт

В `template.json`: схема результата (строгая, `additionalProperties: false`), входы, выходы, при необходимости слот предмета работы.

4 Объявите источники данных

Какие сущности навык читает. Это можно сделать и из интерфейса — карточка навыка, блок «Источники данных».

5 Проверьте схему

```
python scripts/check_templates.py skills/my-skill/template.json
```

6 Доставьте на сервер

Либо пересборкой образа (навыки едут в образе), либо посевом шаблонов без пересборки:

```
python scripts/push_templates.py --base https://<адрес> \
  --user <admin> --password '…' --only my-skill
```

9.4 Правка навыка из интерфейса

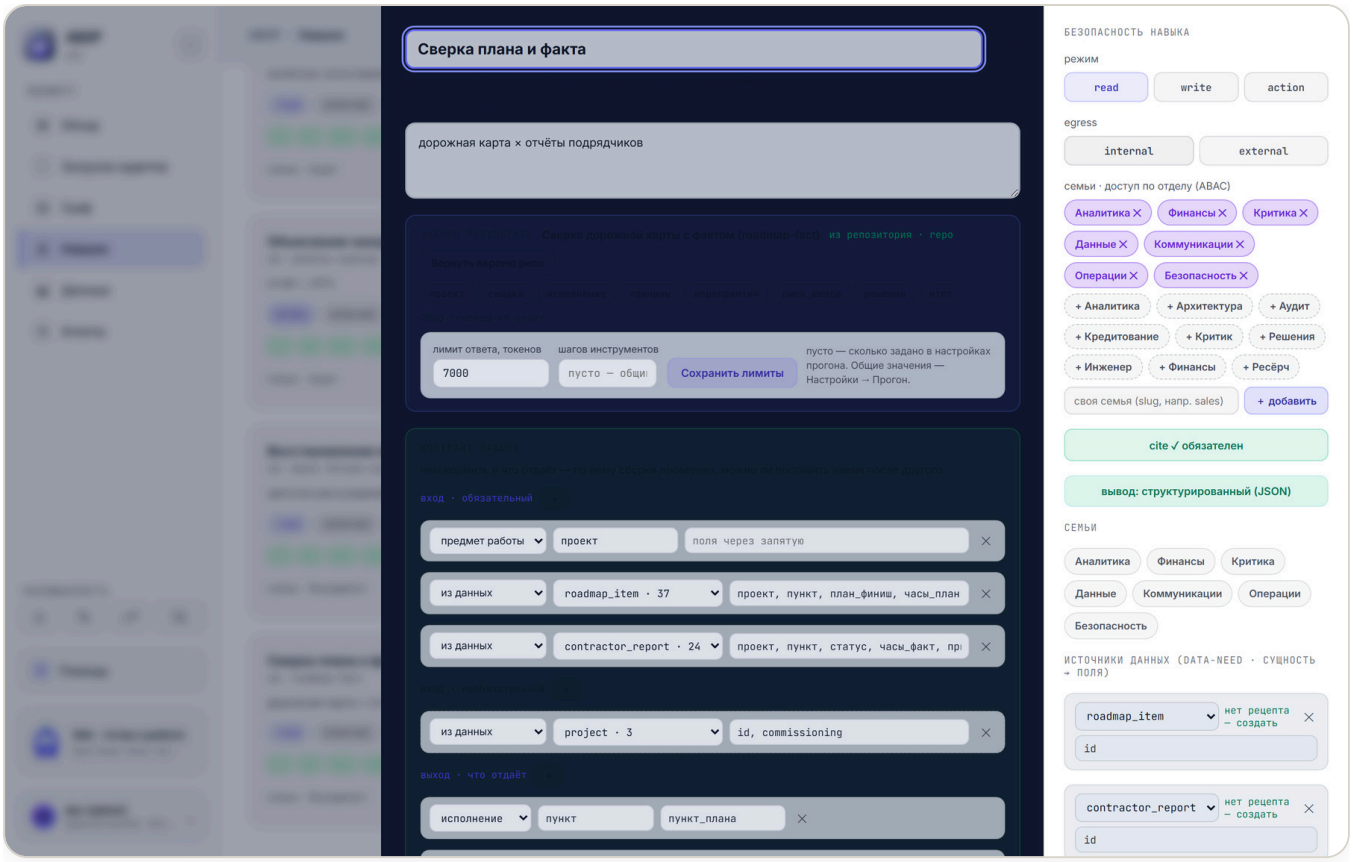


Рис. 9.1. Карточка навыка: назначение, шаблон результата, лимиты, контракт, безопасность, семьи и источники данных — всё правится здесь.

Правки из интерфейса имеют приоритет над поставкой: посев из репозитория больше не перекрывает такой навык. Кнопка «Вернуть версию репо» откатывает навык к поставочному состоянию.

ЧТО ПРАВИТСЯ	КЕМ	ГДЕ
Назначение и описание	admin	карточка навыка
Лимит ответа и шаги инструментов	admin	карточка навыка, блок лимитов
Контракт (вход, выход)	admin	карточка навыка, блок контракта
Источники данных	admin	карточка навыка, правая колонка
Семьи (кому доступен)	admin	карточка навыка, правая колонка
Методика (SKILL.md)	—	только файлом, с пересборкой образа

9.5 Инструменты навыка

Навык может ходить за данными сам — это называется инструментами. Доступные инструменты объявляются в заголовке `SKILL.md`:

`data_query`	запросить записи сущности с фильтром
`data_get`	получить одну запись по идентификатору
`data_schema`	узнать, какие поля есть у сущности
`compute`	посчитать выражение детерминированно, без модели
`chart`	построить график по данным

Число шагов ограничено: по умолчанию два. Каждый шаг — это дополнительный вызов модели, поэтому увеличивать стоит осознанно.

РАЗДЕЛ 10

Агенты и семьи

Агент — навыки, собранные под работу, с именем, отделом и границами. Семья определяет, что агенту доступно.

10.1 Семьи

В поставке десять семей: аналитика, архитектура, аудит, инженер, кредитование, критик, менеджмент, ресёрч, решения, финансы. Семья — это отдел или направление: она задаёт, какие системы и данные видят агенты, и какие навыки им доступны.

Свои семьи заводятся в интерфейсе (карточка навыка, блок «семьи», поле «своя семья») или через API. Идентификатор — латиницей: он же используется как значение атрибута `department` в Keycloak.

10.2 Сборка агента

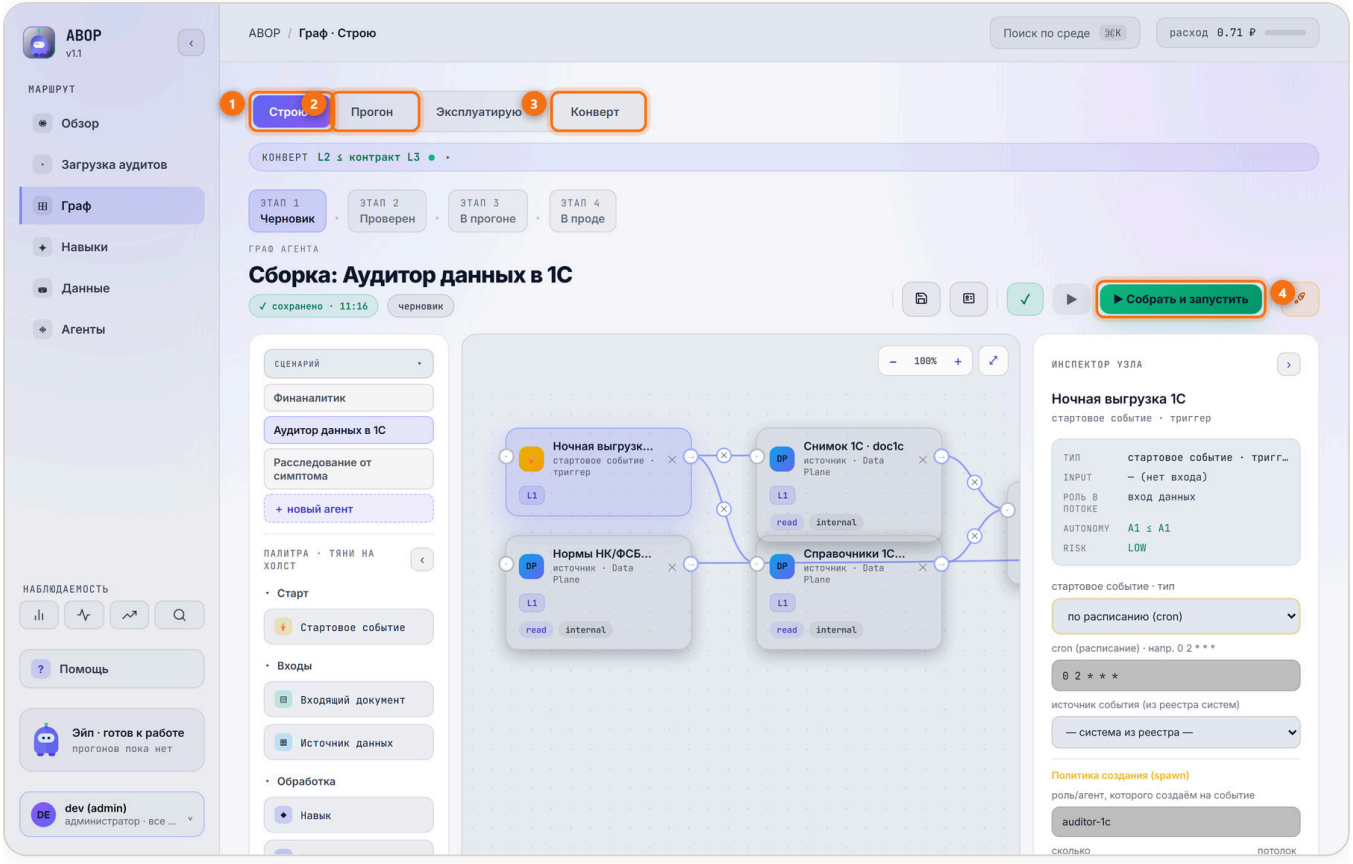


Рис. 10.1. Канва: палитра узлов, холст и инспектор. Слева — сценарии, сверху — режимы и полоса зрелости агента.

Порядок работы: выбрать сценарий или создать нового агента, вытянуть узлы из палитры, соединить, настроить каждый в инспекторе, проверить и сохранить. Проверка перед сохранением не даст собрать агента, нарушающего конверт, — например, поставить узел внешнего действия без подтверждения при недостаточном потолке автономии.

Сборка живёт только здесь. Кнопка « + новый агент» открывает окно задачи, но собирает не «где-то в модалке»: среда разбирает задачу планировщиком по контрактам (раздел 12) и выкладывает результат *на канву* — источники из объявленных сущностей шага, навыки по порядку исполнения, рёбра по объявленным зависимостям `after`. Дальше агент правится руками там же. Построение не запускает прогон и не тратит ни токенов, ни денег, поэтому блок бюджета в этом режиме не показывается вовсе.

Режим «Прогон» показывает прогон, а не его имитацию. Источник один — последний прогон агента, открытого на канве (контракт → агент → `/api/runs`). Нет прогона — пустой экран с кнопкой запуска. Состояние маскота читается с прогона: идёт — думает, очередь подтверждений — ждёт человека, закончился — готово или замечания.

> ЧТО БЫЛО ДО ЭТОГО И ПОЧЕМУ ЭТО ВАЖНО ЗНАТЬ
>
> Лента «Прогон» рисовала ход работы из таймера: каждые 220 мс рос процент, и из этого числа выводилось всё — статусы узлов, этапы, события доски, отметки готовности. Экран выглядел работающим, когда не исполнялось ничего, а четыре места в интерфейсе открывали «пульт прогона демо» вообще без прогона. На показе заказчику это худший вид неправды — показывать занятость вместо работы. Таймер убран вместе с проп-скоростью симуляции. Практическое следствие для поддержки: **если полоса не двигается, работа действительно не идёт** — смотрите очередь (`GET /api/runs/queue`), а не ждите.

10.3 Конверт и автономия

УРОВЕНЬ	ЧТО РАЗРЕШЕНО
A0	только подготовка, ничего не исполняет
A1	чтение данных и выводы
A2	подготовка изменений и документов; наружу — только через подтверждение
A3	исполнение согласованных действий в оговорённых системах
A4	полная автономия в своей области (в поставке не используется)

Конверт задаётся контрактом и проверяется дважды: при сохранении агента и при каждом прогоне. Результат проверки виден в вердикте прогона.

10.4 Жизненный цикл

Черновик → **Проверен** → **В прогоне** → **В проде**. Черновик перезаписывается без создания версий: автосохранение не плодит историю. Осознанный пересмотр создаёт новую версию. Вывод из работы — «В Лимб» (архив): агент недоступен для запуска, но прогоны и находки остаются. Полное удаление доступно администратору и требует подтверждения вводом имени.

РАЗДЕЛ 11

Лимиты и бюджеты

Четыре уровня, от общего к точному. Каждый следующий перекрывает предыдущий, применённые значения видны в результате прогона.

УРОВЕНЬ	ГДЕ ЗАДАЁТСЯ	КТО	НА ЧТО ДЕЙСТВУЕТ
код и окружение	`.env` сервера	администратор сервера	умолчания, если ничего не настроено
среда	Настройки → Прогон	manager и выше	все прогоны всех агентов
навык	карточка навыка	admin	везде, где стоит этот навык
прогон	окно «Новый прогон»	тот, кто запускает	только этот запуск

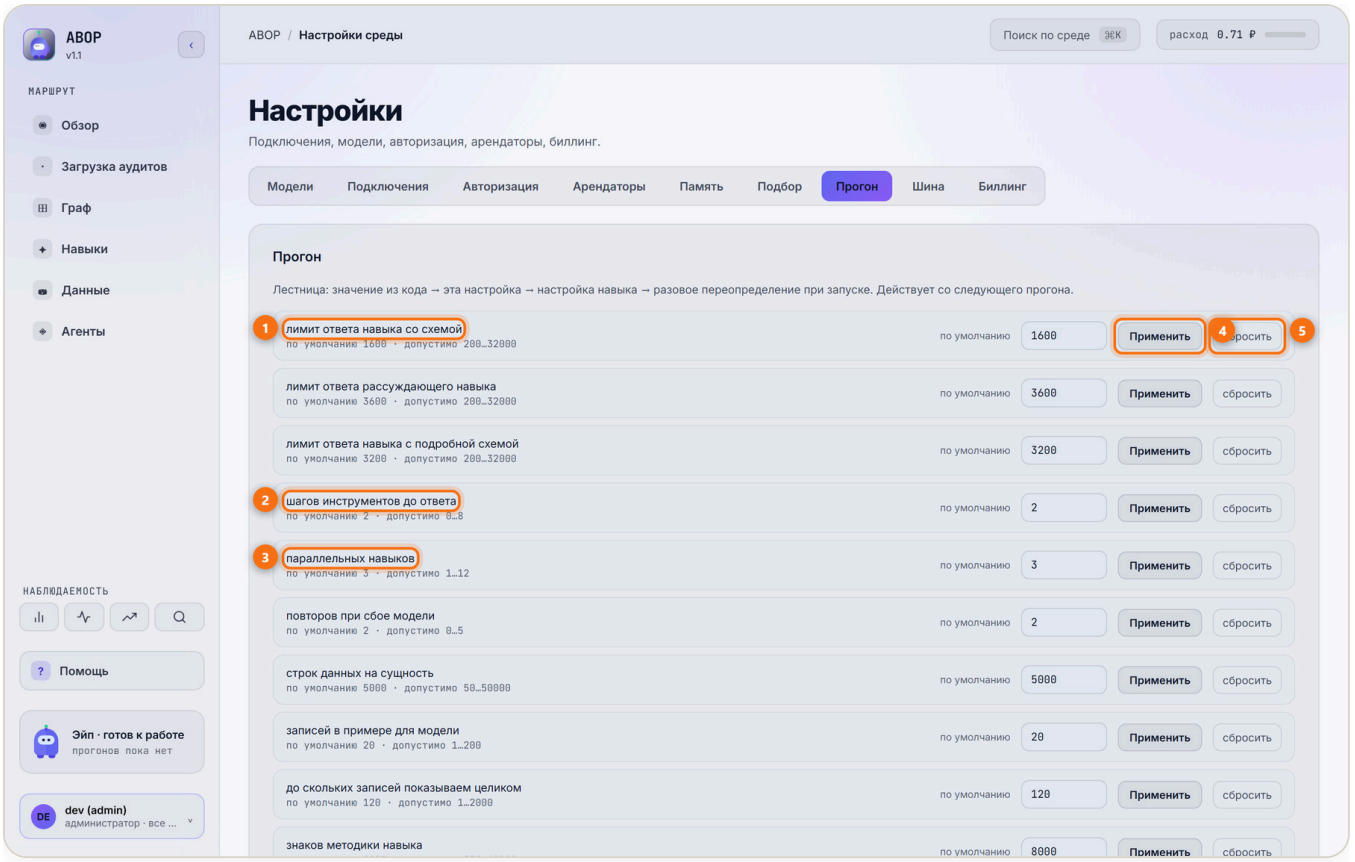


Рис. 11.1. «Настройки → Прогон»: одиннадцать полей с умолчаниями и границами. У каждого видно, задано оно вручную или взято по умолчанию.

1. **Лимит ответа навыка со схемой.** Сколько токенов навык может потратить на структурный ответ.
2. **Шагов инструментов.** Сколько раз навык ходит за данными до ответа; ноль — не ходит.
3. **Параллельных навыков.** Сколько вызовов модели идут одновременно внутри прогона.
4. **Применить.** Настройка действует со следующего прогона, перезапуск не нужен.
5. **Сбросить.** Вернуть поле к умолчанию.

11.1 Одиннадцать настраиваемых полей

ПОЛЕ	УМОЛЧ.	ГРАНИЦЫ	ЧТО ДЕЛАЕТ
лимит ответа навыка со схемой	1600	200… 32000	длина структурного ответа
лимит ответа рассуждающего навыка	3600	200… 32000	длина письма, плана, записки
лимит ответа навыка с подробной схемой	3200	200… 32000	потолок для сложных шаблонов
шагов инструментов до ответа	2	0…8	сколько раз навык ходит за данными
параллельных навыков	3	1…12	одновременные вызовы модели в прогоне
повторов при сбое модели	2	0…5	ретраи при перегрузе и таймауте
строк данных на сущность	5000	50… 50000	сколько записей тянуть для точного счёта
записей в примере для модели	20	1…200	сколько строк уходит в промпт на больших наборах
до скольких записей показываем целиком	120	1…2000	порог, ниже которого модель видит всю выборку
знаков методики навыка	8000	500… 40000	сколько методики попадает в системное сообщение
знаков данных в промпте	20000	500… 60000	потолок блока данных

ЗНАЧЕНИЕ ВНЕ ГРАНИЦ НЕ ОТВЕРГАЕТСЯ МОЛЧА

Оно подрезается до границы, и это видно в поле. Опечатка вроде «200000 токенов» иначе положила бы все прогоны до тех пор, пока кто-нибудь не догадался бы заглянуть в настройки.

11.2 Лимит и бюджет — разные вещи

	ЛИМИТ	БЮДЖЕТ
Ограничивает	один вызов модели	весь прогон
Влияет на	качество: малый лимит обрывает ответ	расход: после превышения новые навыки не запускаются
Единицы	токены ответа, шаги, строки	токены, рубли, секунды

Бюджет проверяется дважды: перед запуском навыка и ещё раз после ожидания в очереди к модели — иначе десять ветвей на двенадцати слотах пробили бы лимит все разом. Превышение показывается честно: расход одного вызова известен только после ответа модели.

РАЗДЕЛ 12

Подбор исполнителя: агент или цепочка навыков

Самый частый вопрос поддержки — «почему выбрался не тот». Этот раздел разбирает подбор целиком: кто принимает решение, по каким порогам, на что он опирается и как его настроить, не ломая остальное. Читать стоит целиком — пороги связаны, и правка одного без понимания остальных даёт обратный эффект.

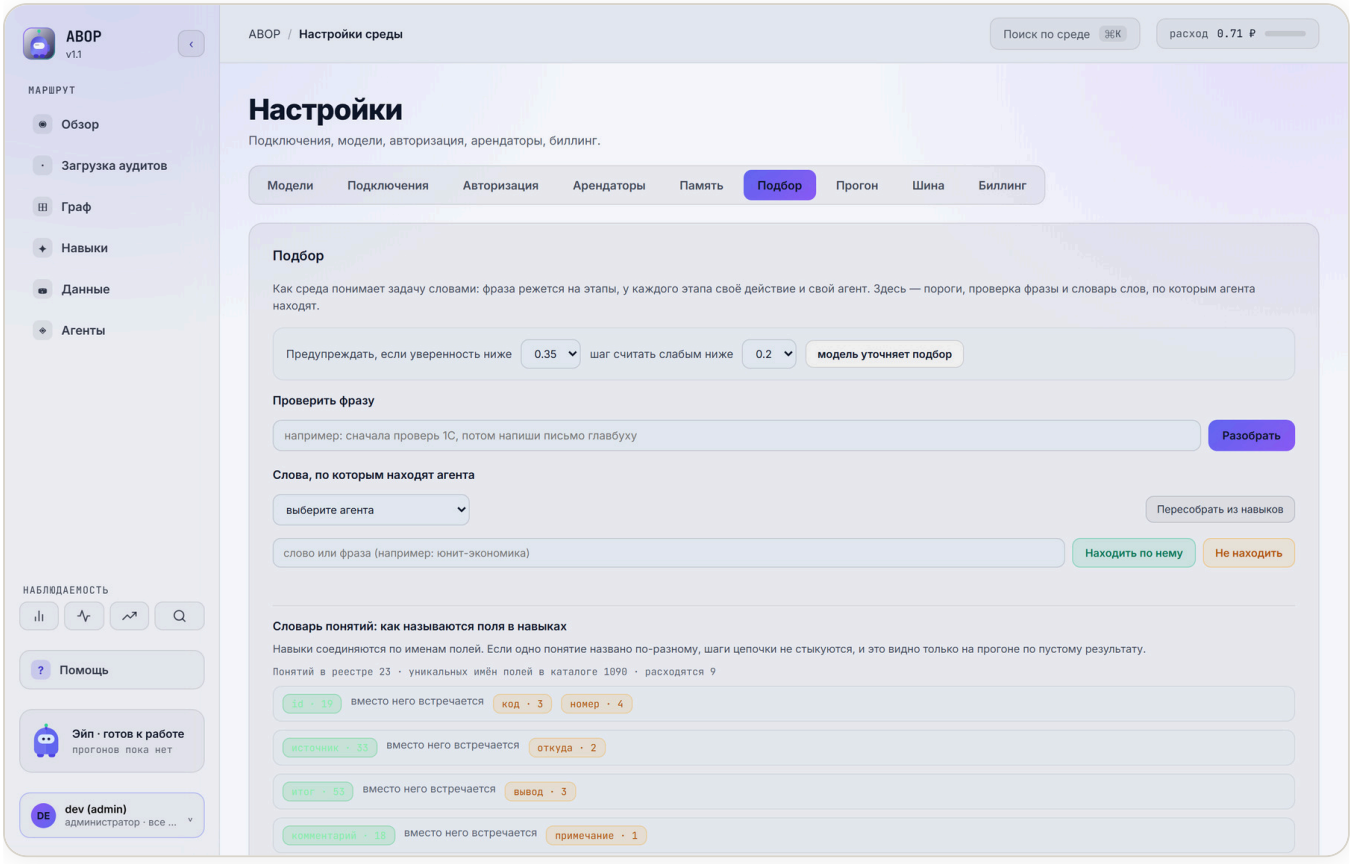


Рис. 12.1. «Настройки → Подбор»: пороги уверенности, песочница разбора фразы и словарь слов, по которым находят агента.

12.1 Решение принимает оркестратор — одно, в одном месте

Раньше каждый канал (чат десктопа, веб, крон, шина) сам сшивал последовательность «достаточно ли описания → есть ли готовый агент → собрать ли из навыков». Каналы расходились: карточка в чате держала план, собранный до правки подбора, и человек запускал устаревший. Теперь решение принимает **оркестратор** — одна ручка `POST /api/orchestrate`, через которую идут все каналы.

Оркестратор *решает, но не действует*: запустить агента или собрать исполнителя вызывающая сторона идёт отдельными ручками. Это не формальность — у действия своя проверка прав и свой журнал.

Исходов ровно четыре:

ИСХОД	КОГДА	ЧТО ВИДИТ ЧЕЛОВЕК
`agent`	Есть готовый агент, уверенно закрывающий задачу	Карточка решения с кнопками запуска
`build`	Готового нет, но по контрактам навыков собирается цепочка	Карточка цепочки с правкой шагов
`ask`	Кандидаты идут рядом, либо в задаче не хватает предмета работы	Уточняющий вопрос с кликабельными кандидатами
`chat`	Это не задача: приветствие, вопрос о среде, просьба объяснить	Обычный ответ модели

Каждое решение возвращает **трассу**: `why[]` — человеческим языком, почему именно так, и `facts{}` — числа, на которых решение построено (счёт лидера, счёт второго, покрытие плана, порог). Трасса ложится в журнал. При разборе жалобы «выбрался не тот» смотреть надо сначала её, а не пороги.

> **ДВА МЕХАНИЗМА ПОДБОРА — ЗАКРЫТАЯ ДЫРА**
>
> До унификации «запустить готового» и «собрать цепочку» были независимыми механизмами: цепочка собиралась из агентов, не знала про навыки, и шаг в ней правился только если нужный агент существовал. Человек видел два разных ответа на одну фразу. Если вы расширяете среду своим каналом — вызывайте `/api/orchestrate`, а не собирайте последовательность заново.

12.2 Пороги выбора готового агента

Константы живут в `server/orchestrator.py`. Менять их в бою стоит только по измеренной причине — и после прогона регресса (12.9).

КОНСТАНТА	ЗНАЧ.	ЧТО ОЗНАЧАЕТ
`AGENT_SURE`	0.32	С этого счёта агент предлагается как уверенный выбор.
`AGENT_FLOOR`	0.25	Ниже — агент не предлагается вовсе: лучше собрать цепочку или спросить, чем выдать наугад.
`AGENT_COVER`	0.6	Доля шагов плана, которую агент должен закрывать. Это *доля*, а не «все шаги»: строгое требование приводило к тому, что среда предлагала второго агента рядом с первым и дублировала работу. Дополнительное условие — агент должен закрывать **ведущий** шаг плана; агент, закрывающий три второстепенных шага из пяти, задачу не делает.
`EDITOR_FROM`	2	С какого числа шагов к цепочке добавляется навык-редактор отчёта: на одном шаге сводить нечего.

Если агентов-кандидатов несколько, берётся лучший по совпадению среди *всех* подходящих, а не первый перешагнувший порог.

12.3 Пороги сборки из навыков

КОНСТАНТА	ЗНАЧ.	ЧТО ОЗНАЧАЕТ
`SKILL_SURE`	0.30	С этого счёта навык считается уверенно подобранным на шаг.
`SKILL_LEAD`	1.4	Во сколько раз лидер должен опережать следующего кандидата, чтобы решение считалось однозначным. Если не опережает — исход `ask`: честнее спросить, чем угадать.
`SKILL_FLOOR`	0.15	Ниже этого счёта отрыв лидера уже ни о чём не говорит — сравнивать нечего.

12.4 Как планировщик строит цепочку

Планировщик (`server/planner.py`) строит цепочку **по контрактам навыков**, а не по догадке модели: выход предыдущего шага должен подходить входу следующего. Что он делает по порядку:

1. **Выделяет указание.** Если к задаче приложен материал (длиннее `_PASTE_FROM` = 600 знаков), указание отделяется от материала: подбор идёт по *указанию*. Без этого вставленное письмо на три экрана разбиралось на одиннадцать «этапов» — по знакам препинания приложенного текста.
2. **Режет на работы.** Не больше `_MAX_STAGES` = 4: больше четырёх работ в одной фразе человек не ставит, остальное — разметка текста. Второй и дальше фрагменты принимаются только если содержат **глагол действия** («сверь», «разбери», «подготовь», «отправь»…).
3. **Убирает ссылки.** Адрес и его подпись («выгрузка: http://…») вырезаются до сопоставления. Иначе имя системы в ссылке притягивало посторонний навык: в финансовой задаче навык аудита 1С всплывал со счётом 0.49 только потому, что в ссылке встречалось слово «1с».
4. **Ранжирует навыки.** По словам (стемминг, `STEM_LEN` = 4), по целевому артефакту (`TARGET_BONUS` = 0.3 — «письмо», «отчёт», «задача» в формулировке), по смыслу (12.5) и по предпочтениям пользователя (12.6). Кандидатов не больше `MAX_CANDIDATES` = 6, счёт ниже `MIN_CANDIDATE` = 0.08 не рассматривается.
5. **Продолжает цепочку по контрактам.** Пока выход последнего шага кому-то подходит со счётом не ниже `CONTINUE_MIN` = 0.12 и шагов не больше `CONTINUE_MAX` = 4. Длина цепочки ограничена сверху ещё и **числом разных глаголов действия** в задаче: «собери дайджест и отправь письмом» — это две работы, и пятого шага там быть не может, даже если контракты сцепляются.
6. **Добавляет редактор.** Навык `report-editor` исключён из ранжирования (он подходит ко всему и забивал бы подбор) и добавляется в конец отдельно, от `EDITOR_FROM` = 2 шагов.

Роль системы в задаче: источник или адресат. Одно и то же слово означает разное. «Разбери входящие *письма*» — почта источник; «напиши *письмо* клиенту» — почта адресат. Словарно навык написания письма выигрывает оба этапа: слово «письмо» стоит у него в каждой второй строке методики. Поэтому роль определяется отдельно и по контрактам:

- **Адресат назван предлогом или глаголом.** «на почту», «в трекер», «в вики» — это `_TARGET_WORDS` . «Напиши письмо», «составь тикет», «оформи страницу» — глагол производства рядом с названием артефакта. Названный адресат даёт навыку, объявившему туда доставку, прибавку `TARGET_BONUS` = 0.3.
- **Предлог источника отменяет «заказанный артефакт».** «Подготовь ответ *по письму* заказчика» — письмо здесь материал. Слова «по», «из», «от», «согласно», «входящие», «полученные» перед артефактом переключают роль. Без этой проверки правило давало прибавку навыкам-письмам ровно там, где просят другой документ, — регресс подбора ловил это на задаче про БФТ.
- **Штраф `SOURCE_ROLE_PENALTY` = 0.35** получает навык, который одновременно: объявил доставку в названную систему, **не** объявил её данные во входах и **не** был назван адресатом. Три условия вместе означают ровно одно: человек назвал систему источником, а навык в неё пишет и её

данных не читает — этап он не делает. Штраф держится крупнее `TARGET_BONUS` намеренно: прямая речь человека о предмете работы сильнее словарного совпадения.

- **Источники считаются по этапу, а не по задаче целиком.** В «разбери письма и напиши письмо-ответ» почта названа источником только в первом этапе. Считать по всей задаче — значит снести навык письма и во втором, где его как раз и просят.

> НЕОБЯЗАТЕЛЬНЫЕ СВЯЗИ
>
> Связь между навыками бывает обязательной и необязательной. Раньше цепочка обрывалась на первой необязательной: формально следующий шаг «не требуется». Теперь она продолжается и по ним, если задача этого просит — именно так финансовая цепочка из шести шагов собирается без ручной правки.

12.5 Смысл: эмбеддинги

Сравнение по смыслу включается поверх словаря и считается через эмбеддинги описаний навыков (кэш в памяти процесса, чтобы не считать одно и то же на каждый запрос).

`SEM_FLOOR`	0.45	Ниже этой близости вклад смысла равен нулю: шум похож на сигнал и сбивает подбор.
`SEM_CAP`	0.25	Потолок прибавки за смысл. Смысл **уточняет** словарь, но не перебивает его: иначе подбор становится необъяснимым, а жалобы — неразбираемыми.

Эмбеддинги — best-effort: если модель эмбеддингов недоступна, подбор работает по словам, а в трассе у шага пустое поле `sem`. Это **не** повод считать, что всё в порядке: пустой `sem` на всех шагах означает, что смысловая часть отключена целиком — проверьте профиль эмбеддингов (раздел 6.3).

12.6 Подбор учится на решениях человека

Таблица `plan_choices` (`server/choice_store.py`) хранит исходы предложений **по каждому пользователю отдельно**: что было предложено, что человек в итоге взял и чем закончилось.

ИСХОД	ВЕС	ОТКУДА ПРИХОДИТ
`accepted`	+1.0	Человек запустил предложенное как есть
`edited`	+1.3	Поправил цепочку руками и запустил — самый сильный сигнал: выбор сделан осознанно
`cancelled`	−1.6	Нажал «Не запускать». Отказ весит больше согласия: согласие бывает от безразличия, отказ — нет

Учитываются последние `WINDOW` = 200 записей, свежие весят больше. Итоговая прибавка нормируется и ограничена `PREFER_CAP` = 0.12 — намеренно мало: прибавка решает спор между двумя похожими навыками, но не назначает исполнителя вместо подбора по существу. Иначе среда замкнулась бы на первых привычках пользователя и перестала бы отвечать на новые формулировки.

Ручки: `POST /api/plan/feedback` (исход предложения; десктоп зовёт её при отказе), `GET /api/plan/prefer` (что выучено по текущему пользователю — первое, что стоит смотреть, если у одного человека подбор ведёт себя иначе, чем у остальных).

12.7 Правило: план строится без подсказок агента

Оркестратор запрашивает план с `hints: false`. Причина — замкнутый круг: подсказка от найденного агента поднимала его же навык в плане, план получался «агентским», и агент после этого «покрывал» план почти полностью. Среда подтверждала сама себе собственную догадку, а слабый подбор выглядел уверенным.

Подсказки (`HINT_WEIGHT` = 0.3 с потолком `HINT_CAP` = 0.1) остаются для вызовов, где плана и агента не сравнивают между собой. Если вы делаете свой канал — не включайте их в сценарии «сначала нашли агента, потом проверяем покрытие планом».

12.8 Настройки подбора в интерфейсе

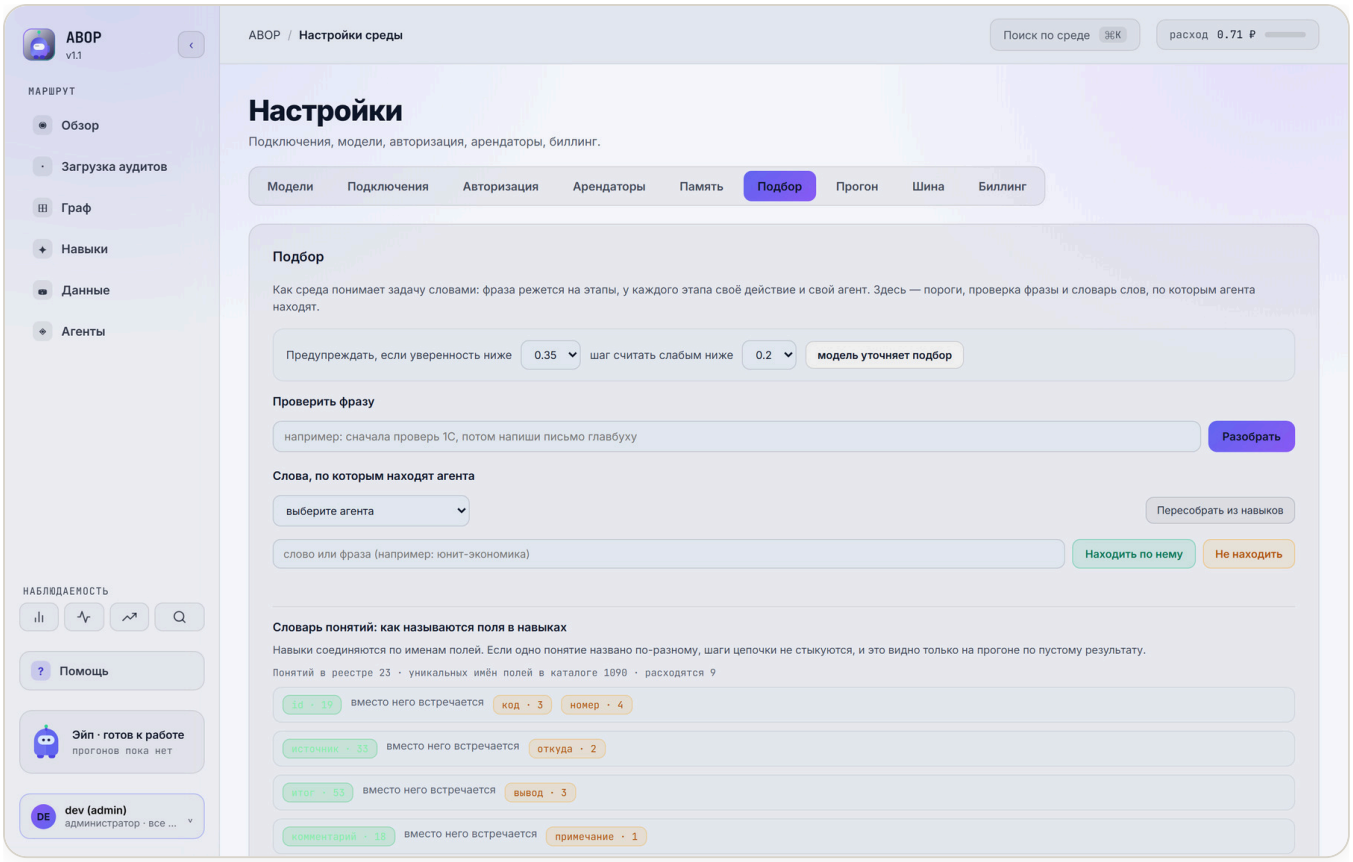


Рис. 12.1. «Настройки → Подбор»: пороги уверенности, песочница разбора фразы и словарь слов, по которым находят агента.

Предупреждать, если уверенность ниже	порог, ниже которого подбор считается сомнительным (по умолчанию 0.35)
Шаг считать слабым ниже	порог для отдельного этапа разбора (0.2)
Уточнение моделью	включать ли сравнение по смыслу. Выключенное — быстрее и дешевле, но хуже на непривычных формулировках

Словарь агента сеется автоматически из имени, роли и навыков и дополняется вручную. Это самый дешёвый и самый предсказуемый инструмент настройки: добавить агенту два слова из речи пользователей лучше, чем сдвинуть общий порог.

12.9 Проверка: песочница и регресс

Песочница. Поле «Проверить фразу» показывает разбор без запуска: на какие работы разложилась задача, какой исполнитель подобран на каждую и с какой уверенностью. Прогоняйте здесь реальные формулировки пользователей после каждого добавления агента.

Регресс. Скрипт `ops/regress_pick.py` гоняет набор задач через ту же ручку, что и десктоп, и печатает таблицу «задача → ожидаемый исход → фактический». Проверяется живой стенд: каталог навыков, реальные агенты и сущности с данными.

```
docker cp ops/regress_pick.py abop-webapi:/app/_rg.py
docker exec abop-webapi python /app/_rg.py
```

Набор покрывает оба случая: задачи, под которые агент есть (ждём `agent`), и задачи, под которые его нет (ждём `build` или `ask`). Ожидания в скрипте — проверенное поведение, а не пожелание: если строка краснеет, сначала смотрите трассу «почему», и только потом правьте ожидание. Рядом лежат юнит-тесты `tests/test_orchestrator_regression.py` — они проверяют правила, а скрипт проверяет, что на настоящем каталоге правила дают ожидаемые решения.

12.10 Разбор жалобы «выбирается не тот»

1. **Трасса.** Откройте `why[]` и `facts{}` этого решения в журнале. В девяти случаях из десяти ответ там: лидер и второй стояли рядом (сработал `SKILL_LEAD`), либо агент не закрывал ведущий шаг (`AGENT_COVER`).
2. **Повторите фразу в песочнице.** Та же формулировка, тот же пользователь.
3. **Посмотрите выученное:** `GET /api/plan/prefer`. Если человек раньше трижды отказался от правильного навыка, подбор это запомнил.
4. **Проверьте смысл.** Пустой `sem` у всех шагов — отключены эмбеддинги.
5. **Правьте в таком порядке:** словарь агента → описание навыка → и только в последнюю очередь общие пороги. Порог меняет поведение всех задач сразу, словарь — одной.

> ПРАКТИКА
>
> Соберите десяток реальных формулировок от пользователей и держите их в `CASES` регресса. Это дешевле, чем разбирать жалобы постфактум, и единственный способ заметить, что правка порога под одну задачу сломала пять других.

12.11 Документы по ссылке

Если в задаче есть ссылка, агент читает документ по ней и опирается на содержимое (`server/doclink.py`). Ограничения и защита:

ПЕРЕМЕННАЯ	ПО УМОЛЧ.	ЧТО ОГРАНИЧИВАЕТ
`ABOP_DOC_MAX`	4	Сколько ссылок читается за один прогон
`ABOP_DOC_MAX_BYTES`	2000000	Размер сырого документа
`ABOP_DOC_MAX_CHARS`	60000	Сколько текста доезжает до навыка
`ABOP_DOC_TIMEOUT`	20	Секунд на загрузку
`ABOP_DOC_HOSTS`	—	Дополнительные разрешённые хосты, через запятую

Что важно для безопасности:

- **Открываются только адреса систем из реестра** (раздел 7) плюс явно перечисленные в `ABOP_DOC_HOSTS`. Сервер стоит внутри периметра, и свободное «загрузи любой адрес» открыло бы агенту доступ к внутренним панелям и метаданным облака.
- **Переадресации проверяются на каждом шаге** (не больше `MAX_HOPS` = 3): разрешённый адрес не должен уводить на неразрешённый.
- **Страница входа документом не считается.** Если вики отдала форму логина, документ помечается непрочитанным, и навыку прямо сказано: ссылаться на него нельзя. Иначе агент делал выводы по тексту «Введите пароль».

Что прочитано и что не прочитано, видно в метриках прогона (`run_metrics.documents`) и в строке «Источник данных» отчёта.

12.12 Длинные прогоны: кэш частей

Многошаговые агенты не укладывались в таймаут запроса, и работа терялась целиком. Теперь готовая часть прогона складывается в таблицу `run_parts` (`server/run_parts_store.py`) по ключу «задание + отпечаток состава агента», и при повторном подходе прогон продолжается с места обрыва, а не с начала. Часть больше `MAX_PART_BYTES` = 256 КБ не кэшируется. После сведения итогового отчёта части удаляются; оставшийся мусор подчищает `sweep`.

Отпечаток состава важен: если агента пересобрали, старые части не подхватятся — иначе отчёт свёлся бы из результатов навыков, которых в агенте больше нет.

РАЗДЕЛ 13

Очередь, группы и веер

Прогоны исполняются не прямо в запросе, а через очередь. Это даёт предсказуемость под нагрузкой и возможность отменить работу.

13.1 Как работает очередь

Воркеры	`ABOP_RUN_WORKERS` потоков внутри процесса разбирают задания
Приоритет	чем меньше число, тем раньше задание; прогоны из чата приоритетнее регулярных
Один на пользователя	`ABOP_USER_CONCURRENT` ограничивает одновременные прогоны одного человека
Таймаут	`ABOP_RUN_TIMEOUT`; задание, не уложившееся в него, помечается честно, а не висит бесконечно
Дедупликация	повторный одинаковый запрос того же пользователя не создаёт второе задание

Состояние очереди — `GET /api/runs/queue`: сколько ждёт, сколько выполняется, среднее время ожидания, занятая память. Те же числа уходят в метрики.

13.2 Группы заданий

Несколько заданий можно объединить в группу — тогда ими управляют как одним целым:

- `GET /api/runs/groups/{id}` — состояние: сколько в каком статусе;
- `POST /api/runs/groups/{id}/cancel` — отменить всю группу;
- `GET /api/runs/groups/{id}/summary` — сводка: результаты сведены по составному ключу, с итогами и разрешением расхождений;
- `POST /api/runs/jobs/{id}/retry-step` — перезапустить одну ветвь, не трогая остальные.

13.3 Веер по предмету

Веер — запуск одного агента сразу по многим предметам: по всем проектам, всем договорам, всем контрагентам. Каждый предмет становится отдельным заданием со своим сужением данных, но с общей доской и общим снимком данных.

```
POST /api/runs/fan-out
{
  "agent_id": "authored-…-контролёр-проектов.v1",
  "slot": "проект",
  "query": "",
  "budget": { "max_tokens": 60000 },
  "limit": 10
}
```

Предметы берутся из явного списка (`values`), подбираются по запросу (`query`) или берутся все доступные записи сущности слота. Размер веера ограничен `АВОР_FANOUT_MAX` .

13.4 Общая память и арбитраж

Ветви одного запроса делят доску: выводы кладутся под ключами вместе с авторством, а читаются только те ключи, которые навык объявил во входах. Если две ветви разошлись в общем поле одной и той же записи, расхождение разрешается по возрастанию цены: правило (большинство, наиболее тяжёлый статус, число по краю, наличие первоисточника) → навык-арбитр → человек.

Спор о сроке, деньгах или предлагаемом мероприятии правилом не закрывается никогда: он остаётся открытым и явно адресован человеку. Нерешённое расхождение видно в записи свода, а не подчищено.

> СНИМОК ДАННЫХ
>
> При запуске веера фиксируется объём и отпечаток выборки. Ветвь, увидевшая другой набор, помечает расхождение данных отдельно от спора ветвей — иначе разные цифры выглядели бы противоречием, хотя считали их по разным данным.

РАЗДЕЛ 14

Шина и коннектор-воркер

Шина нужна для двух вещей: принимать события из ваших систем как триггеры и доставлять команды на исполнение. Без неё платформа работает, но события и внешние действия отключены.

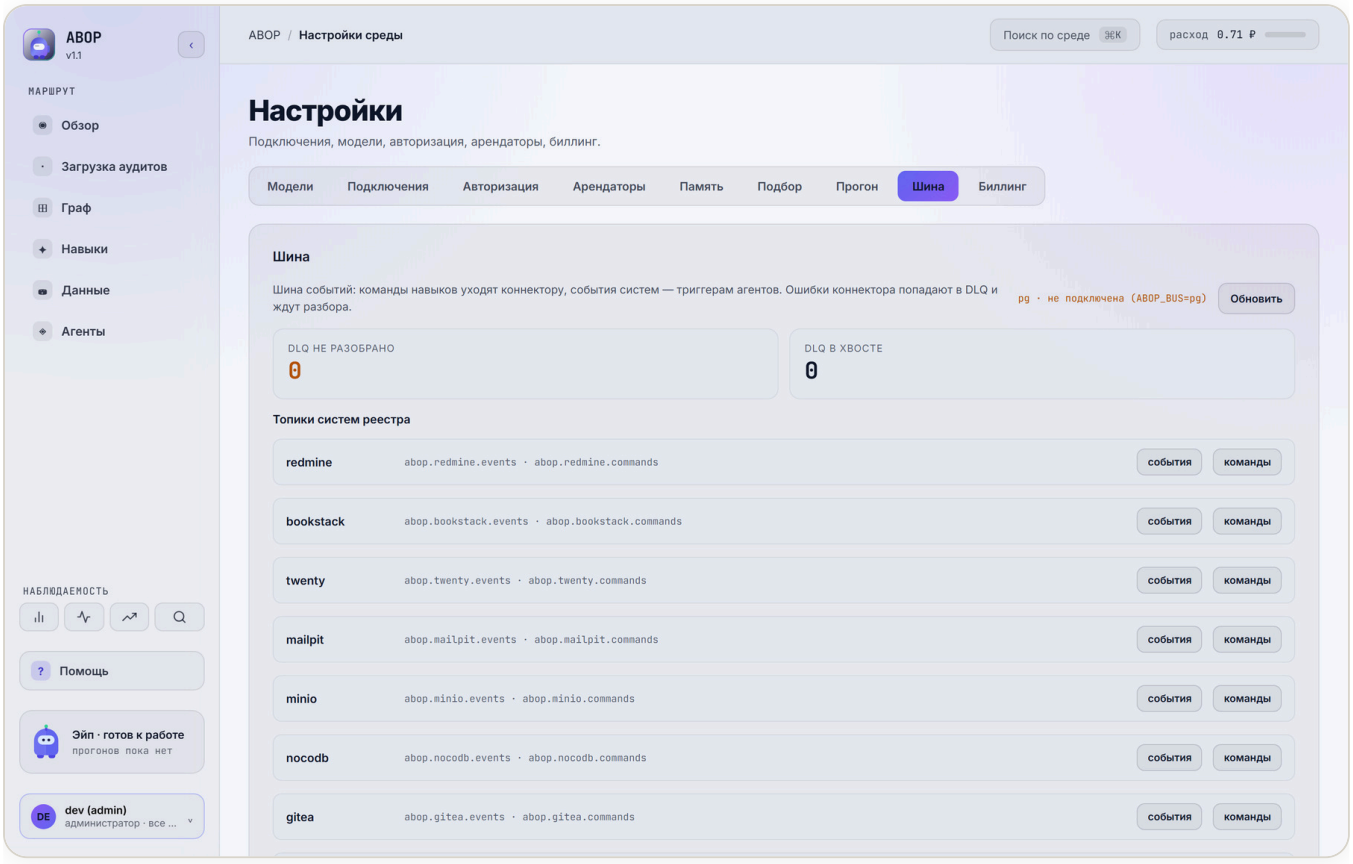


Рис. 14.1. «Настройки → Шина»: топики, поток сообщений и очередь недоставленных.

14.1 Топики

`abop.runs.requests`	задания на прогон
`abop.runs.results`	результаты прогонов
`abop.<система>.events`	события из системы: пришло письмо, создана задача
`abop.<система>.commands`	команды в систему: создать задачу, отправить письмо
`abop.dlq`	недоставленные сообщения

14.2 Подключение

```
ABOP_KAFKA_BROKERS=10.0.0.30:9092
ABOP_KAFKA_SASL_USER=abop
ABOP_KAFKA_SASL_PASSWORD='…'
ABOP_KAFKA_PARTITIONS=3
```

Порты брокера открывайте только для сервера приложения и коннектор-воркера.

14.3 Коннектор-воркер

Отдельный процесс, который читает топики команд и исполняет их в ваших системах. У него — секреты доступа на запись; у ядра ABOP их нет. Воркер ведёт собственный учёт исполненных команд, поэтому повторная доставка одного сообщения не создаёт вторую задачу.

Здоровье воркера: `/healthz` и `/metrics` на его порту. Метрики `abop_connector_{consumed,done,failed,dlq,duplicate}` показывают, что он делает.

14.4 Очередь недоставленных

Сообщение, которое не удалось обработать, уходит в `abop.dlq` с указанием причины. Разбор — из интерфейса или через API:

- `GET /api/bus/dlq` — что лежит и почему;
- `POST /api/bus/dlq/replay` — повторить обработку после устранения причины;
- `POST /api/bus/dlq/ack` — признать сообщение разобранным.

СЛЕДИТЕ ЗА DLQ

Растущая очередь недоставленных означает, что часть работы агентов не доезжает до систем. Заведите оповещение на `abop_bus_dlq_total`.

РАЗДЕЛ 15

Доставка и шаблоны отчётов

Куда уходит результат навыка, описывается в самом навыке — секцией `delivery`. Платформа ничего не отправляет сама: команда становится заявкой на подтверждение.

15.1 Секция доставки

```
"delivery": {
  "system": "redmine",              // система из реестра
  "type":   "issue.create",         // тип команды коннектора
  "each":   "находки",              // массив → по команде на элемент
  "where":  { "ранг": ["критично", "существенно"] },
  "limit":  10,                       // максимум команд за прогон
  "title":  "Задача: {{заголовок}}",  // заголовок карточки подтверждения
  "payload": {
    "subject":     "Находка {{id}}: {{описание}}",
    "description": "{{$.итог}}\n\n{{#действия}}",
    "project":     "abop"
  }
}
```

ПОДСТАНОВКА	ЧТО БЕРЁТ
`{{путь.к.полю}}`	значение из элемента (или из корня, если `each` не задан)
`{{$.путь}}`	значение из корня ответа навыка
`{{#путь}}`	список маркированными строками

Списки склеиваются через «; », объекты — как «ключ: значение». Ограничение `limit` обязательно для массовых доставок: без него одна находка на сотню строк превращается в сотню задач.

15.2 Путь команды

```
навык отдал результат
        │
        ▼
  сборка команды по секции delivery
        │
        ▼
  заявка на подтверждение  ──► человек видит ПОЛНЫЙ текст того, что уйдёт
        │ «разрешить»
        ▼
  публикация в топик команд ──► коннектор-воркер ──► ваша система
        │
        ▼
  событие с результатом (id, ссылка) ──► обратно в карточку прогона
```

15.3 Шаблоны отчётов

Отчёт — это HTML-шаблон с подстановками, из которого собирается документ и, при наличии сервиса рендера, PDF. Шаблоны лежат в `reports/<идентификатор>.html` и сеются в базу при старте; правка в веб-редакторе имеет приоритет и переживает обновление.

ПОДСТАНОВКА	ЧТО ПОДСТАВЛЯЕТСЯ
`{{title}}`, `{{agent}}`, `{{date}}`	шапка отчёта
`{{verdict}}`	итог проверки прогона
`{{findings_total}}`, `{{investigations_total}}`	количества
`{{findings}}`	таблица находок

РАЗДЕЛ 16

Аудит и безопасность

Три контура защиты: кто и что может (RBAC), к каким данным допущен (ABAC) и что может уйти наружу (подтверждение).

16.1 Что закрыто по умолчанию

Аутентификация	Без настроенного Keycloak API отвечает 401. Открытого режима в продуктивной сборке нет
Внешние действия	Ничего не уходит без подтверждения человека. Это свойство платформы, а не настройка агента
Данные вне области	Сущность из системы, недоступной семье агента, не читается; отказ пишется в аудит
Ввод пользователя	Задача человека, приложенные файлы, данные и наблюдения инструментов приходят модели помеченными блоками данных. Инструкции внутри них не исполняются — методика и роль не меняются
Выход в сеть	Запросы к внутренним и служебным адресам запрещены, если система не объявлена внутренней
Персональные данные	Маскировка задаётся правилом рецепта (`pii_mask`) и применяется до попадания записи в хранилище

16.2 Проверка перед сдачей заказчику

1 **Убедитесь, что режим разработки выключен**

```
docker exec abop-webapi printenv | grep -i dev_auth   # должно быть пусто
curl -si https://<адрес>/api/agents | head -1          # 401 без токена
```

2 **Проверьте области систем**

В реестре у каждой системы должен быть явный список семей. Пустой список означает «доступна всем» — это осознанное решение, а не умолчание по невнимательности.

3 **Проверьте учётные записи коннекторов**

Читающие — только на чтение. Пишущие — только в нужный проект или раздел.

4 **Смените пароли по умолчанию**

Панель наблюдаемости, база, брокер шины.

5 **Закройте порты**

Наружу — только обратный прокси. Postgres, Keycloak, брокер и воркер недоступны из внешней сети.

6 **Проверьте аудит**

Запустите прогон, подтвердите внешнее действие, откройте «Админ-панель → Аудит ИБ»: обе операции должны быть в списке с вашим именем.

16.3 Штатное расписание

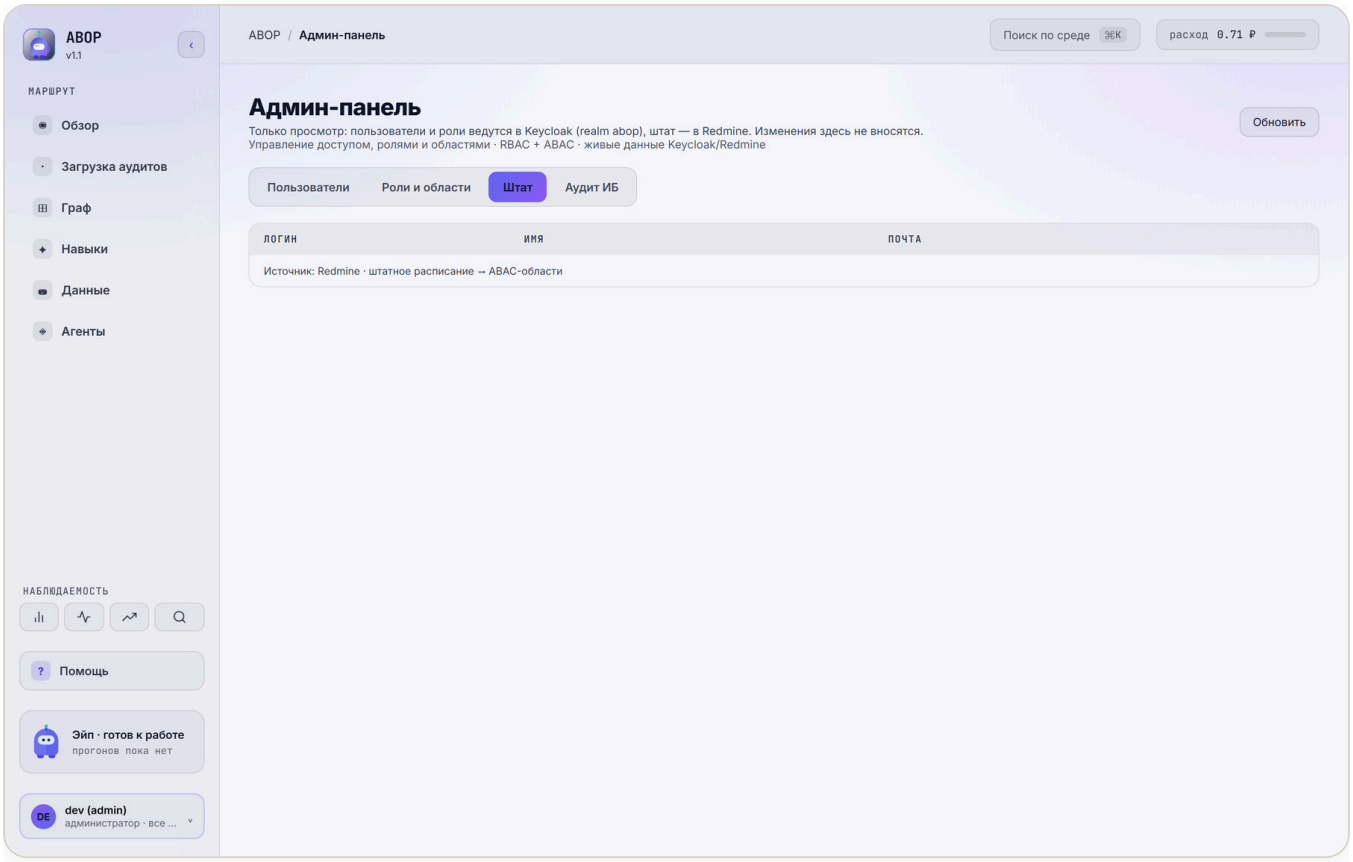


Рис. 16.1. «Штат»: соответствие людей и учётных записей — по нему видно, чьи действия стоят за каждым прогоном.

РАЗДЕЛ 17

Наблюдаемость

Метрики в формате Prometheus отдаются по `/metrics`. Панель с готовым дашбордом поднимается из состава поставки.

```
cd /opt/abop
docker compose -f ops/docker-compose.observability.yml up -d
```

17.1 Что смотреть каждый день

МЕТРИКА	О ЧЁМ ГОВОРИТ	КОГДА ТРЕВОЖНО
`abop_runs_total{ok}`	прогоны: успешные и с замечаниями	доля неуспешных растёт
`abop_run_seconds`	длительность прогона	p95 уходит к таймауту
`abop_run_queue_depth`	сколько заданий ждёт	очередь не рассасывается — мало воркеров или ёмкости модели
`abop_run_cost_rub_total`	расход в деньгах	резкий рост при неизменной нагрузке
`abop_llm_fallback_total`	откаты на запасную модель	любое ненулевое значение — основная модель нестабильна
`abop_run_soft_errors_total`	нефатальные сбои шагов	растёт — что-то отваливается тихо
`abop_bus_dlq_total`	недоставленные сообщения	любое ненулевое — часть работы не доехала
`abop_arbitration_human_total`	расхождения, ждущие человека	растёт — источники данных разъехались

17.2 Сквозная трасса

У каждого прогона есть идентификатор трассы; он же передаётся в сообщениях шины заголовком. По нему собирается полный путь: запрос → задание → прогон → команда → исполнение в системе. Ветви веера ссылаются на родительскую трассу, поэтому картина «один запрос → двадцать ветвей» собирается целиком.

17.3 Журнал событий

Процесс пишет структурированный журнал: одна строка — одно событие в JSON. Полезные события: `agent.run.done`, `run_limits.changed`, `board.save_failed`, `fanout.started`, `skill_schemas.bound`.

```
docker logs -f abop-webapi | grep -v '"level":"debug"'
```

РАЗДЕЛ 18

Биллинг и квоты

Стоимость считается по фактическим токенам ответа модели и тарифу, а не оценочно.

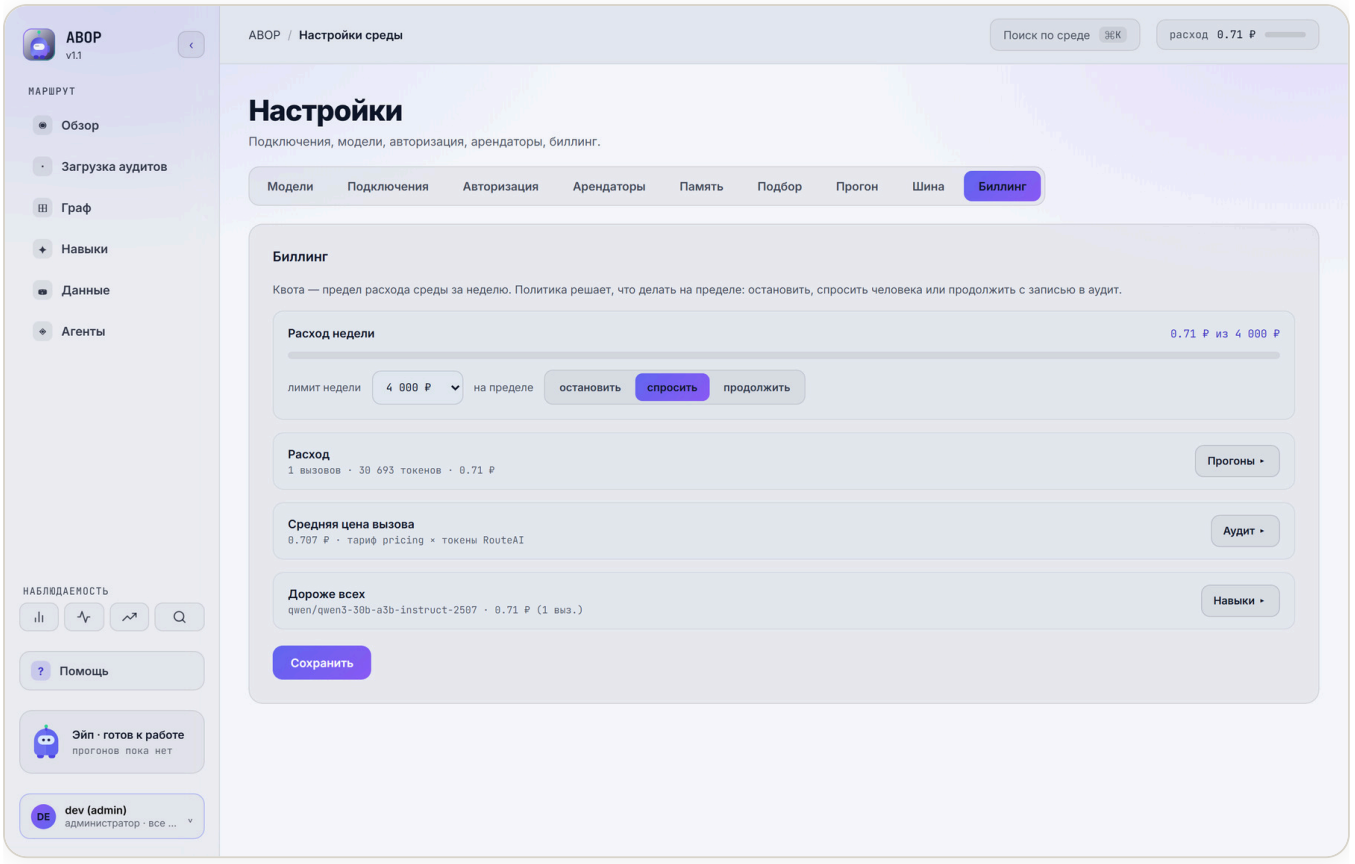


Рис. 18.1. «Настройки → Биллинг»: израсходовано за период, самые дорогие навыки, средняя цена вызова.

18.1 Как считается

После каждого вызова модели фиксируются входные и выходные токены. Стоимость получается умножением на тариф модели. Если тариф не задан (например, своя модель на своём железе), расход считается нулевым — и это честно: денег действительно не тратится.

18.2 Квоты

Квота токенов	задаётся в админ-настройках; видна пользователю в шапке и в кабинете
Мягкий предел	70% — счётчик желтеет, 90% — краснеет
Бюджет прогона	дополнительная защита на уровне одного запуска (раздел 11)

18.3 Как снизить расход

- **Сужайте предмет работы.** Прогон по одному проекту дешевле прогона по всем сразу в разы.
- **Уменьшите «записей в примере для модели»** на больших наборах: истину считает код, модель объясняет — ей не нужны все строки.
- **Уменьшите шаги инструментов** у навыков, которым не нужен поиск: каждый шаг — отдельный вызов модели.
- **Включите кэш прогонов** (`АВОР_RUN_CACHE=1`): повторный одинаковый запрос вернёт сохранённый результат.
- **Свой бокс.** На собственном железе расход в деньгах равен нулю; ограничением становится время, а не бюджет.

РАЗДЕЛ 19

Резервное копирование и обновление

Данные АВОР живут в двух местах. Потерять можно и то и другое — бэкап нужен для обоих.

19.1 Что где лежит

ГДЕ	ЧТО	ПОТЕРЯ ОЗНАЧАЕТ
Postgres	Агенты, контракты, прогоны и очередь, шаблоны навыков и отчётов, определения рецептов и коннекторов, настройки, доступы, память, заявки на подтверждение, разметка находок	Потерю всей истории и всех настроек
Том `abop_ape`	Канонические записи Data Plane (`/root/.ape/data/*.jsonl`), файловые копии рецептов	Потерю выгруженных данных — их придётся собрать рецептами заново
Том коннектора	Учёт исполненных команд (защита от дублей)	Риск повторного исполнения команд при переигровке очереди

Внутри Postgres стоит отдельно знать про таблицы, появившиеся позже основной схемы: они создаются сами при старте, но ведут себя при потере по-разному.

ТАБЛИЦА	ЧТО ХРАНИТ	ПОТЕРЯ ОЗНАЧАЕТ
`chat_threads`, `chat_messages`	Разговоры десктопа. История живёт на сервере, а не в приложении — поэтому она видна с любой машины и сохраняется при переустановке клиента	Пропажу всей переписки пользователей. Бэкапить обязательно
`plan_choices`	Исходы предложений подбора по каждому пользователю (раздел 12.6)	Подбор «забудет» привычки людей и вернётся к базовому ранжированию. Не критично, но заметно
`board_entries`	Факты на общей доске с авторством	Потерю долгой памяти отделов. Бэкапить обязательно
`run_parts`	Готовые части длинных прогонов (раздел 12.12)	Ничего: незавершённый прогон начнётся заново. Таблицу можно чистить руками

19.2 Резервное копирование

```
# база
docker exec abop-pg pg_dump -U abop abop | gzip > /backup/abop-$(date +%F).sql.gz

# том с каноническими данными
docker run --rm -v abop_ape:/src -v /backup:/dst alpine \
  tar czf /dst/abop_ape-$(date +%F).tar.gz -C /src .
```

Проверяйте восстановление раз в квартал: бэкап, который никогда не восстанавливали, — это надежда, а не резервная копия.

19.3 Восстановление

```
# база
gunzip -c /backup/abop-2026-09-30.sql.gz | docker exec -i abop-pg psql -U abop abop

# том
docker run --rm -v abop_ape:/dst -v /backup:/src alpine \
  sh -c "cd /dst && tar xzf /src/abop_ape-2026-09-30.tar.gz"
```

19.4 Обновление

1. **Снимите бэкап**

   Оба хранилища, прямо перед обновлением.

2. **Убедитесь, что очередь пуста**

   ```
   curl -s http://127.0.0.1:8091/api/runs/queue
   ```

   Обновление во время работающего прогона приведёт к его перезапуску.

3. **Разложите новую поставку и пересоберите**

   ```
   cd /opt/abop && tar -xf abop-<новая версия>.tar && bash ops/deploy-webapi.sh
   ```

4. **Проверьте здоровье и каталог**

   ```
   curl -s http://127.0.0.1:8091/api/health
   ```

   Число навыков не должно уменьшиться.

5. **Прогоните проверочный прогон**

   Любой лёгкий агент с бюджетом «короткий»: убедитесь, что модель отвечает и данные читаются.

> **ОТКАТ**
>
> Предыдущий образ остаётся в докере. Откат — пересоздать контейнер из него с тем же томом и восстановить базу из бэкапа, если менялись схемы.

19.5 Обновление шаблонов навыков без пересборки

Рантайм читает шаблоны из базы, репозиторий — источник посева. Обновить шаблоны на работающей системе:

```
python scripts/check_templates.py skills/*/template.json
python scripts/push_templates.py --base https://<адрес> --user <admin> --password
'…'
```

Навыки, правленные из интерфейса, без флага `--force` не перезаписываются.

РАЗДЕЛ 20

Диагностика

Таблица симптомов. В каждом случае начинайте с идентификатора прогона: по нему поднимается весь след.

СИМПТОМ	ПРИЧИНА	ЧТО ПРОВЕРИТЬ
Прогон отрабатывает за секунду, находок ноль, модель не звалась	Data Plane пуст: том не смонтирован или рецепты не прогоняли	`docker inspect abop-webapi --format '{{json .Mounts}}' | grep abop_ape`, затем `docker exec abop-webapi wc -l /root/.ape/data/*.jsonl`. Пусто — прогнать рецепты
API отвечает 401 на всё	Не настроена или сломалась проверка токенов	`KEYCLOAK_JWKS_URI` доступен из контейнера; issuer совпадает с тем, что в токене; часы сервера синхронны
«свой бокс не отвечает», прогоны уходят в облако	Модель недоступна по заданному адресу	`GET /api/llm/status`; `curl` к `/v1/models` модели с сервера приложения; адрес в «Настройки → Модели»
Очередь растёт, прогоны ждут	Мало воркеров или ёмкости модели	`GET /api/runs/queue`; поднять `ABOP_RUN_WORKERS` и `ABOP_LLM_MAX_INFLIGHT` согласованно с ёмкостью бокса
Ответы навыков обрываются на полуслове	Мал лимит ответа	«Настройки → Прогон» — лимит ответа; для одного навыка — его карточка. В результате прогона видно, какие лимиты применялись
Навык вернул не те поля	Навык привязан к чужому шаблону результата или схема не строгая	Карточка навыка: «шаблон результата» — тот ли. В результате прогона поле `schema_miss` перечисляет незаполненные обязательные поля
Результат смешал несколько предметов	Предмет работы не задан, данные пришли по всем	У навыка должен быть объявлен слот. Проверьте `GET /api/skills/{id}/slots`
Внешние действия не исполняются после подтверждения	Коннектор-воркер не работает или нет доступа к системе	`/healthz` воркера; `GET /api/bus/dlq`; права учётной записи в целевой системе

СИМПТОМ	ПРИЧИНА	ЧТО ПРОВЕРИТЬ
Задания выполняются дважды	Обновление сервиса во время работающего прогона	Обновляйтесь при пустой очереди; задание с `attempts>1` в журнале это подтвердит
Контейнер перезапускается по памяти	Слишком много воркеров при заданном лимите памяти	`abop_process_rss_mb`; уменьшить `ABOP_RUN_WORKERS` или поднять `--memory`
В редакторе контракта не та сущность	Сущность есть в данных, но не в списке — устаревший клиент	`GET /api/data/entities` должен её возвращать; обновите страницу с очисткой кэша

20.1 Быстрая проверка установки

```
# ядро живо и каталог на месте
curl -s http://127.0.0.1:8091/api/health

# модель отвечает
curl -s http://127.0.0.1:8091/api/llm/status

# данные есть
curl -s http://127.0.0.1:8091/api/data/entities

# очередь разгружена
curl -s http://127.0.0.1:8091/api/runs/queue

# метрики отдаются
curl -s http://127.0.0.1:8091/metrics | head -20
```

РАЗДЕЛ 21

Карта процессов: редакции, области, подтверждение

Карта — общий документ организации: её правят несколько человек, применяют целиком и сверяют с журналом прогонов. Этот раздел о том, как она устроена в базе, кто что вправе менять и что делать при переносе со старого стенда.

21.1 Что где лежит

Карта и назначения раньше хранились в `app_config` двумя JSON-полями. Теперь это отдельный стор `server/process_store.py` с пятью таблицами:

`process_map`	применённая редакция дерева: `(area, revision, tree, applied_by, applied_at)`. Карта организации одна, она лежит под областью `*`; слой AS-IS читается отсюда
`process_seats`	посадка агента на ОДНУ операцию: `(op_id, audit_id, pin_version, area, since, until)`. Ссылка на агента без версии; снятие — `until`, не удаление строки
`process_revisions`	редакция: `(area, base, status, version, author, approved_by)`. Статусы: черновик → на согласовании → применена / отклонена. Уникальный индекс держит ровно одну открытую редакцию на область
`process_changes`	журнал правок редакции, append-only: `(rev_id, seq, kind, target, payload, author)`. TO-BE — это применённое дерево с наложенным журналом, отдельно он нигде не хранится
`process_presence`	кто сейчас смотрит область. Таблица, а не память процесса: при нескольких репликах список верен

Поток процесса хранится рёбрами внутри контейнера (`edges: [{from, to}]`), положение блока — его `x/y`. Связи живут внутри одного уровня зума. Карта, записанная до перехода на явный поток,

нормализуется при чтении: порядок детей превращается в ту же цепочку.

21.2 Области и права

Область объявляется на процессе первого уровня (`area`) и наследуется вглубь. Правило доступа отличается от обычного ABAC: **читают все, правит только своя область**.

analyst	видит карту целиком, не правит ничего
manager	правит ветки своей области (`department == area`)
support	видит всё сквозь области, процессы отдела не правит
admin	правит любую область и верхнюю рамку карты (область `*`); подписывает редакции, требующие согласования

> Область правки сервер берёт из самой карты по цели правки, а не из тела запроса: подменить `area` в запросе бесполезно. Редакция живёт в одной области — пачка правок, задевающая две, отбивается с перечислением узлов.

21.3 Согласование

Обычную редакцию применяет тот, кто её правил. Подпись требуется, когда префлайт упирается в человека:

- на участке **закреплена версия агента** (`pin_version`) — смена версии на регулируемом участке должна быть событием, а не побочным эффектом;
- агент **действует наружу** (узел `out` или egress `external`) — ADR-014.

Тогда «Применить» отбивается с причиной, менеджер жмёт «Отправить на согласование» (`POST /api/process/revision/{id}/submit`), и редакция закрывается для правок: подписант подпишет ровно то, что прочёл. Применяет её `admin` — и только ту, что ему отправили: применить «за себя» то, что требует согласования, нельзя даже администратору.

Префлайт применения дополнительно проверяет, что агент есть в реестре и его автономия не выше потолка контракта LUDA (ADR-013). Применение переводит посаженных агентов из черновика в `deployed` — иначе карта утверждала бы, что агент работает участок, пока он не принимает триггеров.

21.4 Одновременная правка

Журнал правок дописывается под `SELECT … FOR UPDATE`, поэтому число реплик API не влияет на целостность. У каждой правки считаются три множества: что она пишет, что ей нужно, что она рушит. Правки сливаются, если множества не пересеклись; иначе приходит `409` с именем того, кто опередил. Удаление узла рушит и его поддерево — поэтому правка внутрь удаляемого подпроцесса ловится, а не повисает в пустоте.

Между репликами оповещение идёт существующим `cachebus` (Postgres LISTEN/NOTIFY, топик `process`) — отдельной шины для карты нет. Клиент держит SSE `GET /api/process/stream?area=` и берёт разницу по `?since=`.

21.5 Подтверждение прогонами

Прогон, запущенный триггером, запоминает операции участка (`op_ids` в payload) — их берут из посадок агента. Ручной запуск с канвы идёт без них: это и есть признак «вне карты».

`GET /api/process/liveness?window=24h|7d|30d` возвращает по каждому участку число прогонов за окно, дату последнего и состояние (`подтверждён` / `тихо` / `осиротел`), а также агентов, которые работают, но ни на одной операции не сидят. Признак выводится, а не хранится — как и статус развёртывания.

21.6 Перенос со старого стенда

Если карта и назначения остались в `app_config`, перенесите их скриптом. Сухой прогон по умолчанию, запись — флагом:

```
docker exec abop-webapi python scripts/migrate_process_map.py
docker exec abop-webapi python scripts/migrate_process_map.py --apply
docker exec abop-webapi python scripts/migrate_process_map.py --area
"Закупки=management" --apply
```

Что делает перенос: дерево становится применённой редакцией, назначения разворачиваются в посадки по одной на операцию, `layer: AS-IS` уходит в факт, `layer: TO-BE` — в открытый черновик. Агент ищется в реестре **по имени**; не нашёлся — участок остаётся пустым, и скрипт перечисляет такие назначения. Это намеренно: лучше видеть пустой участок, чем посаженного неизвестно кого.

Перенос проходит тем же хуком развёртывания, что и обычное применение редакции: посаженный агент-черновик переводится в `deployed`, приостановленный остаётся на паузе. Если карту переносили сборкой, где этого ещё не было, посаженные агенты могли остаться черновиками — проверьте их статус во Флоте и разверните вручную.

> Области процессов задаёт тот, кто переносит: от них зависит, какой отдел сможет править какую ветку. Значения по умолчанию заданы в самом скрипте и перекрываются `--area "Процесс=область"`. Область должна существовать среди семей (`/api/families`), иначе процесс останется под `*` и править его сможет только администратор.

После переноса ключи `app_config.tree` и `app_config.assignments` сервером не читаются — их можно удалить, проверив карту в интерфейсе.

РАЗДЕЛ 22

Приложение А. Эндпоинты API

Основные группы. Все требуют токена, кроме `/api/health` и `/metrics`. Уровень доступа указан там, где он выше аналитика.

Состояние и служебное

`GET /api/health`	живость, число семей, навыков, адаптеров
`GET /metrics`	метрики Prometheus
`GET /api/llm/status`	адрес модели, каскад, результат проверки
`GET /api/me`	кто вы: уровень, отдел, роли
`GET /api/observability`	сводка наблюдаемости
`GET /api/guides`	руководства, доступные в этой установке

Карта процессов

`GET /api/process/map?layer=as-is|to-be&area=`	срез карты: дерево, посадки, номер редакции, черновик, области, кто сейчас правит
`POST /api/process/map`	записать карту целиком новой применённой редакцией — вход для сида и переноса. **admin**
`POST /api/process/revision`	дописать правки в черновик своей области; сливает с чужими, возвращает пропущенное. **manager+**
`GET /api/process/revision/{id}/changes?since=`	разница по номеру
`DELETE /api/process/revision/{id}`	отбросить черновик. **manager+**
`POST /api/process/revision/{id}/submit`	на согласование, когда префлайт требует человека. **manager+**
`POST /api/process/revision/{id}/apply`	применить: префлайт, одна транзакция на дерево и посадки, деплой посаженных агентов. **manager+**, подпись — **admin**
`GET /api/process/liveness?window=`	подтверждение участков прогонами и агенты вне карты
`GET /api/process/stream?area=`	SSE: живые изменения карты и присутствие
`GET /api/process/graph`	граф-слой регламента по ключу НСИ
`POST /api/process/conformance`	сверка собранного процесса с регламентом

Коды ответа карты: `409` — конфликт правок, в теле номер версии и кто опередил; `422` — отказ по состоянию карты с причиной текстом; `403` — чужая область с перечислением узлов. Причина всегда в теле — интерфейс её показывает.

Прогоны

`POST /api/runs`	запустить прогон; тело: агент, контекст, предмет, бюджет, лимиты
`GET /api/runs`	журнал прогонов
`GET /api/runs/{id}`	прогон целиком: находки, доска, метрики
`GET /api/runs/{id}/findings`	находки отдельно
`GET /api/runs/{id}/report`	отчёт по шаблону
`GET /api/runs/{id}/diff`	отличия от предыдущего прогона агента
`GET /api/runs/{id}/board`	доска прогона: выводы с авторством
`GET /api/runs/search`	поиск внутри прогонов по содержимому
`GET /api/runs/queue`	состояние очереди
`POST /api/runs/jobs/{id}/cancel`	отменить задание

Группы и веер

`POST /api/runs/fan-out`	запуск по многим предметам (manager+)
`GET /api/runs/groups/{id}`	состояние группы
`GET /api/runs/groups/{id}/summary`	сводка с итогами и арбитражем
`GET /api/runs/groups/{id}/board`	общая доска группы
`POST /api/runs/groups/{id}/cancel`	отменить группу (manager+)
`POST /api/runs/jobs/{id}/retry-step`	перезапустить одну ветвь

Агенты и навыки

`GET /api/agents`	список агентов
`POST /api/agents`	сохранить агента (manager+)
`POST /api/agents/author`	собрать агента из набора навыков (manager+)
`POST /api/agents/match`	подобрать агента по фразе
`GET /api/agents/{id}/slots`	какие предметы работы нужны агенту
`POST /api/agents/{id}/retire`	в архив (manager+)
`GET /api/skills`	каталог навыков
`GET /api/skills/{id}/contract`	контракт навыка
`POST /api/schema-templates/{id}`	править шаблон навыка (admin)
`POST /api/schema-templates/{id}/reset`	вернуть версию поставки (admin)
`GET /api/catalog/coverage`	покрытие каталога контрактами

Данные

`GET /api/data/entities`	сущности: реестр, записи в хранилище, использование навыками
`GET /api/data/recipes`	рецепты
`POST /api/data/recipes/{name}/run`	прогнать рецепт
`POST /api/data/recipe/preview`	предпросмотр без записи
`GET /api/data/connectors`	коннекторы
`POST /api/data/connectors/test`	проверить подключение
`GET /api/data/query/{entity}`	прочитать канонические записи
`GET /api/data/lineage`	карта: источник → сущность → потребитель
`GET /api/resolve/{entity}`	подобрать предмет работы по запросу

Настройки и доступ

`GET /api/admin/run-limits`	действующие лимиты, умолчания, границы
`POST /api/admin/run-limits`	изменить лимиты (manager+)
`GET/POST /api/admin/llm`	адрес и модель
`GET/POST /api/admin/config`	настройки среды (manager+)
`GET /api/admin/users`	пользователи из Keycloak (admin/support)
`GET /api/admin/audit`	аудит-след (admin/support)
`GET /api/systems`	реестр систем
`GET /api/billing`	расход и квоты

Подбор, план и доска

`POST /api/orchestrate`	**одно решение по задаче**: кого звать, что собрать, какие границы — и трасса «почему». Через неё идут все каналы
`POST /api/plan/auto`	план из навыков по контрактам; поля `hints`, `semantic`, `prefer`, `max_steps`
`POST /api/plan/auto/build`	собрать исполнителя по плану; вместе с планом принимает `task`, `offered`, `manual` — на них учится подбор
`POST /api/plan/feedback`	исход предложения: `accepted` / `edited` / `cancelled`
`GET /api/plan/prefer`	что подбор выучил по текущему пользователю
`GET /api/board`	общая доска: факты пользователя и его отдела
`POST /api/board/facts`	вынести факт на доску (автор — человек)
`DELETE /api/board/facts/{key}`	снять факт с доски
`POST /api/runs/{id}/to-board`	вынести итоги прогона на доску

Подтверждения и шина

`GET /api/hitl/queue`	очередь подтверждений
`POST /api/hitl/{id}/approve`	разрешить или отклонить (manager+)
`GET /api/bus`	состояние шины
`GET /api/bus/dlq`	недоставленные сообщения
`POST /api/bus/dlq/replay`	повторить обработку (admin)

РАЗДЕЛ 23

Приложение Б. Переменные окружения

Обязательные

ПЕРЕМЕННАЯ	УМОЛЧ.	НАЗНАЧЕНИЕ
POSTGRES_DSN	—	строка подключения к базе
KEYCLOAK_ISSUER	—	адрес realm
KEYCLOAK_JWKS_URI	—	адрес ключей проверки токена
KEYCLOAK_AUDIENCE	course-mcp	ожидаемый получатель токена

Модель

LOCAL_LLM_BASE_URL	—	адрес своей модели (OpenAI-совместимый)
LOCAL_LLM_MODEL	—	имя модели
LOCAL_LLM_API_KEY	EMPTY	ключ, если требуется
ROUTEAI_BASE_URL	—	адрес облачного шлюза
ROUTEAI_API_KEY	—	ключ шлюза
ROUTEAI_STANDARD_CASCADE	—	каскад моделей для агентов
ABOP_LLM_TIMEOUT	120	таймаут запроса к модели, с
ABOP_LOCAL_LLM_TIMEOUT	300	таймаут для своей модели, с
ABOP_LLM_MAX_INFLIGHT	12	общий потолок одновременных запросов

Исполнение прогонов

ABOP_RUN_WORKERS	2	воркеров очереди
ABOP_RUN_TIMEOUT	600	потолок задания, с
ABOP_RUN_STALE_SEC	таймаут+120	когда задание считается зависшим
ABOP_USER_CONCURRENT	1	одновременных прогонов на пользователя
ABOP_USER_RATE_MAX	30	запусков в окне
ABOP_USER_RATE_WINDOW	300	окно ограничения, с
ABOP_RUN_CACHE	1	кэш одинаковых прогонов
ABOP_RUN_MEM_SOFT_MB	1500	мягкий предел памяти процесса
ABOP_FANOUT_MAX	20	максимум заданий в веере

Умолчания лимитов (перекрываются настройками)

ABOP_RUN_LLM_CONCURRENCY	3	параллельных навыков в прогоне
ABOP_RUN_LLM_RETRIES	2	повторов при сбое модели
ABOP_RUN_LLM_BACKOFF	1.5	база задержки между повторами
ABOP_RUN_LLM_TRUNCATE	1	**на проде 0** — иначе промпты режутся
ABOP_RUN_MAX_TOKENS	1600	лимит структурного ответа
ABOP_RUN_MAX_TOKENS_FREE	3600	лимит свободного ответа
ABOP_RUN_MAX_TOKENS_TEMPLATE	3200	лимит для подробных схем
ABOP_RUN_STRUCTURED	1	строгий структурный вывод модели
ABOP_TOOL_STEPS	2	шагов инструментов у навыка

Документы по ссылке

ABOP_DOC_MAX	4	сколько ссылок читается за прогон
ABOP_DOC_MAX_BYTES	2000000	размер сырого документа, байт
ABOP_DOC_MAX_CHARS	60000	сколько текста доезжает до навыка
ABOP_DOC_TIMEOUT	20	таймаут загрузки, с
ABOP_DOC_HOSTS	—	дополнительные разрешённые хосты через запятую (кроме систем из реестра)

Шина и планировщик

ABOP_BUS	–	включение шины
ABOP_KAFKA_BROKERS	–	адреса брокеров
ABOP_KAFKA_SASL_USER / _PASSWORD	–	учётные данные брокера
ABOP_KAFKA_PARTITIONS	3	партиций у создаваемых топиков
ABOP_KAFKA_TOPIC_REQUESTS	abop.runs.requests	топик заданий
ABOP_KAFKA_TOPIC_RESULTS	abop.runs.results	топик результатов
ABOP_KAFKA_TOPIC_DLQ	abop.dlq	недоставленные
ABOP_SCHEDULER	1	планировщик регулярных запусков
ABOP_SCHEDULER_TICK	45	период опроса расписаний, с
ABOP_SCHEDULER_MIN_INTERVAL	60	минимальный интервал между запусками, с
ABOP_SCHEDULER_MAX_FIRES	3	сколько расписаний срабатывает за тик

Прочее

`ABOP_DEV_AUTH`	–	**только разработка.** Отключает проверку токенов
`ABOP_EXTRA_JWKS`	–	дополнительные наборы ключей (несколько realm)
`ABOP_JWT_CACHE_TTL`	`60`	время жизни кэша проверенных токенов, с
`MINIO_* / STORAGE_LIMIT_BYTES`	–	объектное хранилище для отчётов
`LANGFUSE_*`	–	трассировка вызовов модели
`LOG_LEVEL`	`INFO`	уровень журнала

РАЗДЕЛ 24

Приложение В. Порты

СЕРВИС	ПОРТ	КОМУ ОТКРЫВАТЬ
АВОР Web API	8091	Обратному прокси. Наружу — только через TLS
Postgres	5432 / 5433	Только петля сервера приложения
Keycloak	8080 / 8811	Прокси или внутренняя сеть; наружу не обязателен — вход идёт через АВОР
Модель (vLLM)	8000	Только сервер приложения
Брокер шины	9092	Только сервер приложения и коннектор-воркер
Админ брокера	9644	Только сбор метрик
Коннектор-воркер	9105	Только сбор метрик и петля
Prometheus	9091	Внутренняя сеть
Grafana	3300	Внутренняя сеть; сменить пароль по умолчанию
Рендер PDF	3050	Только сервер приложения
Объектное хранилище	9000	Сервер приложения; публичные ссылки — через прокси
Десктоп-клиент (локально)	8799	Только петля на машине пользователя

СОПУТСТВУЮЩИЕ ДОКУМЕНТЫ

«АВОР · Руководство пользователя» — как работать с платформой каждый день. В составе поставки также: описание архитектуры, регламент контрактов навыков, описание общей памяти и арбитража, справочник лимитов прогона.